

Line-by-Line Mathematical Guide for HAETAE_ROM_Programming.ec

Generated HAETAE EasyCrypt proof guide

2026-05-11

File Role

Counted random-oracle programming events and bounds.

How To Read This Script

Read this file as the fundamental-lemma infrastructure for the ROM: define clear/bad states, prove preservation, then bound the bad-event probability.

Line-by-Line Mathematical Reading

Each row preserves one EasyCrypt source line and gives the intended mathematical or proof-script reading. Blank rows are kept because they mark proof structure in long EasyCrypt developments.

Line	EasyCrypt source	Mathematical reading
1	<code>require import AllCore Real Distr List FSet PROM StdOrder Mu_mem.</code>	Imports AllCore Real Distr List FSet PROM StdOrder Mu_mem. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
2	<code>require import HAETAE_Params HAETAE_Algebra HAETAE_Distributions.</code>	Imports HAETAE_Params HAETAE_Algebra HAETAE_Distributions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
3	<code>require import HAETAE_ROM HAETAE_Transcript.</code>	Imports HAETAE_ROM HAETAE_Transcript. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
4	<code>require import HAETAE_Rejection.</code>	Imports HAETAE_Rejection. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
5	<code>require import HAETAE_Reductions.</code>	Imports HAETAE_Reductions. Mathematically, this exposes earlier carrier sets, operations, games, and lemmas that the current file may cite.
6	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
7	<code>theory HAETAE_ROM_Programming.</code>	Begins the namespace HAETAE_ROM_Programming, grouping the declarations as one checked theory.
8	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
9	<code>import HAETAE_Params.</code>	Opens HAETAE_Params so its names can be used without qualification in the following proof.
10	<code>import HAETAE_Algebra.</code>	Opens HAETAE_Algebra so its names can be used without qualification in the following proof.
11	<code>import HAETAE_Distributions.</code>	Opens HAETAE_Distributions so its names can be used without qualification in the following proof.
12	<code>import HAETAE_ROM.</code>	Opens HAETAE_ROM so its names can be used without qualification in the following proof.
13	<code>import HAETAE_Transcript.</code>	Opens HAETAE_Transcript so its names can be used without qualification in the following proof.

Line	EasyCrypt source	Mathematical reading
14	<code>import HAETAE_Rejection.</code>	Opens HAETAE_Rejection so its names can be used without qualification in the following proof.
15	<code>import HAETAE_Reductions.</code>	Opens HAETAE_Reductions so its names can be used without qualification in the following proof.
16	<code>import RealOrder.</code>	Opens RealOrder so its names can be used without qualification in the following proof.
17	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
18	<code>type programming_site = ro_query * ro_output.</code>	Declares carrier set programming_site. EasyCrypt will type-check all later functions and games over this mathematical domain.
19	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
20	<code>op programming_site_query (site : programming_site) : ro_query = site.'1.</code>	Defines operation programming_site_query, a total mathematical function or abbreviation used by later declarations.
21	<code>op programming_site_output (site : programming_site) : ro_output = site.'2.</code>	Defines operation programming_site_output, a total mathematical function or abbreviation used by later declarations.
22	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
23	<code>op programming_site_of_transcript</code>	Defines operation programming_site_of_transcript, a total mathematical function or abbreviation used by later declarations.
24	<code>(md : mode) (tr : transcript) (y : ro_output) : programming_site =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
25	<code>(transcript_challenge_query md tr, y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
26	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
27	<code>op signature_programming_site_of_transcript</code>	Defines operation signature_programming_site_of_transcript, a total mathematical function or abbreviation used by later declarations.
28	<code>(md : mode) (tr : transcript) (y : ro_output) : programming_site =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
29	<code>(transcript_signature_challenge_query md tr, y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
30	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
31	<code>op transcript_signature_challenge_output (tr : transcript) : ro_output =</code>	Defines operation transcript_signature_challenge_output, a total mathematical function or abbreviation used by later declarations.
32	<code>ro_output_of_challenge (sig_challenge (transcript_signature tr)).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
33	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
34	<code>op actual_signature_programming_site</code>	Defines operation actual_signature_programming_site, a total mathematical function or abbreviation used by later declarations.
35	<code>(md : mode) (tr : transcript) : programming_site =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
36	<code>signature_programming_site_of_transcript md tr</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
37	<code>(transcript_signature_challenge_output tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
38	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
39	<code>op honest_signing_programming_site</code>	Defines operation honest_signing_programming_site, a total mathematical function or abbreviation used by later declarations.
40	<code>(md : mode) (sk : skey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
41	<code>(coins : random_coins) : programming_site =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
42	<code>actual_signature_programming_site md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
43	<code>(transcript_from_honest_signing md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
44	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
45	<code>op honest_signing_programming_site_query</code>	Defines operation honest_signing_programming_site_query, a total mathematical function or abbreviation used by later declarations.
46	<code>(md : mode) (sk : skey) (m : message) (ctx : context)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
47	(coins : random_coins) : ro_query =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
48	programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
49	(honest_signing_programming_site md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
50	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
51	op programmed_site_query_min_entropy_bound_for	Defines operation programmed_site_query_min_entropy_bound_for, a total mathematical function or abbreviation used by later declarations.
52	(d : random_coins distr)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
53	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
54	(qs : ro_query list) (bd : real) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
55	forall q, q \in qs =>	Part of a universal mathematical condition that must hold for all listed values.
56	mu d	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
57	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
58	honest_signing_programming_site_query md sk m ctx coins = q) <= bd.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
59	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
60	op programmed_site_query_min_entropy_bound	Defines operation programmed_site_query_min_entropy_bound, a total mathematical function or abbreviation used by later declarations.
61	(md : mode) (sk : skey) (m : message) (ctx : context)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
62	(qs : ro_query list) (bd : real) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
63	programmed_site_query_min_entropy_bound_for drandom_coins md sk m ctx qs bd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
64	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
65	op programming_site_matches_transcript	Defines operation programming_site_matches_transcript, a total mathematical function or abbreviation used by later declarations.
66	(md : mode) (tr : transcript) (site : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
67	programming_site_query site = transcript_challenge_query md tr /\	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
68	transcript_challenge_matches tr (programming_site_output site).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
69	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
70	op reprogramming_failure	Defines operation reprogramming_failure, a total mathematical function or abbreviation used by later declarations.
71	(md : mode) (tr : transcript) (site : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
72	! programming_site_matches_transcript md tr site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
73	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
74	op challenge_entropy_support_ok (md : mode) (tr : transcript) : bool =	Defines predicate challenge_entropy_support_ok, an event or well-formedness condition used in later implications and game proofs.
75	challenge_sparse md (transcript_challenge tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
76	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
77	op min_entropy_failure (md : mode) (tr : transcript) : bool =	Defines predicate min_entropy_failure, an event or well-formedness condition used in later implications and game proofs.
78	! challenge_entropy_support_ok md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
79	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
80	op transcript_log_min_entropy_clear (md : mode) (trs : transcript list) : bool =	Defines predicate transcript_log_min_entropy_clear, an event or well-formedness condition used in later implications and game proofs.
81	all (fun tr => ! min_entropy_failure md tr) trs.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
82	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
83	op signing_record_log_min_entropy_clear	Defines operation signing_record_log_min_entropy_clear, a total mathematical function or abbreviation used by later declarations.
84	(md : mode) (rs : signing_transcript_record list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
85	all (fun r => ! min_entropy_failure md (signing_record_transcript r)) rs.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
86	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
87	op fs_with_aborts_failure	Defines operation fs_with_aborts_failure, a total mathematical function or abbreviation used by later declarations.
88	(md : mode) (tr : transcript) (site : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
89	reprogramming_failure md tr site \/ min_entropy_failure md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
90	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
91	op transcript_signature_programming_site_clear	Defines operation transcript_signature_programming_site_clear, a total mathematical function or abbreviation used by later declarations.
92	(md : mode) (tr : transcript) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
93	! fs_with_aborts_failure md tr (actual_signature_programming_site md tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
94	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
95	op transcript_log_signature_programming_sites_clear	Defines operation transcript_log_signature_programming_sites_clear, a total mathematical function or abbreviation used by later declarations.
96	(md : mode) (trs : transcript list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
97	all (transcript_signature_programming_site_clear md) trs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
98	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
99	op fs_with_aborts_bound_obligation : bool =	Defines predicate fs_with_aborts_bound_obligation, an event or well-formedness condition used in later implications and game proofs.
100	0%r <= fs_with_aborts_reprogramming_term /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
101	0%r <= fs_with_aborts_min_entropy_term.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
102	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
103	type counted_rom_event =	Declares carrier set counted_rom_event. EasyCrypt will type-check all later functions and games over this mathematical domain.
104	[CountedHashQuery of ro_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
105	CountedSignatureSite of programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
106	CountedProgrammedSite of programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
107	CountedMinEntropyCheck of transcript].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
108	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
109	op counted_event_is_hash_query (ev : counted_rom_event) : bool =	Defines predicate counted_event_is_hash_query, an event or well-formedness condition used in later implications and game proofs.
110	with ev = CountedHashQuery _ => true	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
111	with ev = CountedSignatureSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
112	with ev = CountedProgrammedSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
113	with ev = CountedMinEntropyCheck _ => false.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
114	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
115	op counted_event_is_signature_site (ev : counted_rom_event) : bool =	Defines predicate counted_event_is_signature_site, an event or well-formedness condition used in later implications and game proofs.
116	with ev = CountedHashQuery _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
117	with ev = CountedSignatureSite _ => true	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
118	with ev = CountedProgrammedSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
119	with ev = CountedMinEntropyCheck _ => false.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
120	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
121	op counted_event_is_programmed_site (ev : counted_rom_event) : bool =	Defines predicate counted_event_is_programmed_site, an event or well-formedness condition used in later implications and game proofs.
122	with ev = CountedHashQuery _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
123	with ev = CountedSignatureSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
124	with ev = CountedProgrammedSite _ => true	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
125	with ev = CountedMinEntropyCheck _ => false.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
126	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
127	op counted_event_is_min_entropy_check (ev : counted_rom_event) : bool =	Defines predicate counted_event_is_min_entropy_check, an event or well-formedness condition used in later implications and game proofs.
128	with ev = CountedHashQuery _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
129	with ev = CountedSignatureSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
130	with ev = CountedProgrammedSite _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
131	with ev = CountedMinEntropyCheck _ => true.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
132	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
133	op ro_query_is_challenge (q : ro_query) : bool =	Defines predicate ro_query_is_challenge, an event or well-formedness condition used in later implications and game proofs.
134	with q = MessageHashQuery _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
135	with q = ChallengeHashQuery _ => true	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
136	with q = MatrixExpandQuery _ => false	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
137	with q = SamplerExpandQuery _ => false.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
138	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
139	op challenge_query_log_count (qs : ro_query list) : int =	Defines integer-valued parameter challenge_query_log_count, used for dimensions, budgets, support sizes, or bounds.
140	card (oflist (filter ro_query_is_challenge qs)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
141	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
142	lemma challenge_query_log_count_cons_le q qs :	States checked lemma challenge_query_log_count_cons_le. The following proof script must derive this proposition from prior definitions and lemmas.
143	challenge_query_log_count (q :: qs) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
144	challenge_query_log_count qs + 1.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
145	proof.	Begins the proof script for the preceding proposition.
146	rewrite /challenge_query_log_count.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
147	case q => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
148	+ smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
149	+ move=> site.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
150	rewrite oflist_cons fsetUC fcardU1.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
151	smt().	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
152	+ smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
153	+ smt().	Solves a proof branch produced by a previous split, transitivity, or case analysis.
154	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
155	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
156	lemma challenge_query_log_count_nonchallenge_cons q qs :	States checked lemma challenge_query_log_count_nonchallenge_cons. The following proof script must derive this proposition from prior definitions and lemmas.
157	! ro_query_is_challenge q =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
158	challenge_query_log_count (q :: qs) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
159	challenge_query_log_count qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
160	proof.	Begins the proof script for the preceding proposition.
161	rewrite /challenge_query_log_count.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
162	case q => /=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
163	+ by done.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
164	+ by done.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
165	+ by done.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
166	+ by done.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
167	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
168	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
169	lemma challenge_query_log_count_message_cons pk ctx m qs :	States checked lemma challenge_query_log_count_message_cons. The following proof script must derive this proposition from prior definitions and lemmas.
170	challenge_query_log_count (message_hash_query pk ctx m :: qs) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
171	challenge_query_log_count qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
172	proof.	Begins the proof script for the preceding proposition.
173	apply challenge_query_log_count_nonchallenge_cons.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
174	by rewrite /message_hash_query /ro_query_is_challenge.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
175	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
176	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
177	lemma challenge_query_log_count_matrix_cons md sd qs :	States checked lemma challenge_query_log_count_matrix_cons. The following proof script must derive this proposition from prior definitions and lemmas.
178	challenge_query_log_count (matrix_expand_query md sd :: qs) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
179	challenge_query_log_count qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
180	proof.	Begins the proof script for the preceding proposition.
181	apply challenge_query_log_count_nonchallenge_cons.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
182	by rewrite /matrix_expand_query /ro_query_is_challenge.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
183	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
184	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
185	lemma challenge_query_log_count_sampler_cons coins qs :	States checked lemma challenge_query_log_count_sampler_cons. The following proof script must derive this proposition from prior definitions and lemmas.
186	challenge_query_log_count (sampler_expand_query coins :: qs) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
187	challenge_query_log_count qs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
188	proof.	Begins the proof script for the preceding proposition.
189	apply challenge_query_log_count_nonchallenge_cons.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
190	by rewrite /sampler_expand_query /ro_query_is_challenge.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
191	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
192	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
193	op counted_trace_hash_queries (evs : counted_rom_event list) : int =	Defines integer-valued parameter counted_trace_hash_queries, used for dimensions, budgets, support sizes, or bounds.
194	size (filter counted_event_is_hash_query evs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
195	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
196	op counted_trace_signature_sites (evs : counted_rom_event list) : int =	Defines integer-valued parameter counted_trace_signature_sites, used for dimensions, budgets, support sizes, or bounds.
197	size (filter counted_event_is_signature_site evs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
198	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
199	op counted_trace_programmed_sites (evs : counted_rom_event list) : int =	Defines integer-valued parameter counted_trace_programmed_sites, used for dimensions, budgets, support sizes, or bounds.
200	size (filter counted_event_is_programmed_site evs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
201	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
202	op counted_trace_min_entropy_checks (evs : counted_rom_event list) : int =	Defines integer-valued parameter counted_trace_min_entropy_checks, used for dimensions, budgets, support sizes, or bounds.
203	size (filter counted_event_is_min_entropy_check evs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
204	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
205	op programming_sites_same_query	Defines operation programming_sites_same_query, a total mathematical function or abbreviation used by later declarations.
206	(left right : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
207	programming_site_query left = programming_site_query right.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
208	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
209	op programming_sites_conflict	Defines operation programming_sites_conflict, a total mathematical function or abbreviation used by later declarations.
210	(left right : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
211	programming_sites_same_query left right /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
212	programming_site_output left <> programming_site_output right.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
213	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
214	op programming_site_prequeried	Defines operation programming_site_prequeried, a total mathematical function or abbreviation used by later declarations.
215	(queries : ro_query list) (site : programming_site) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.

Line	EasyCrypt source	Mathematical reading
216	<code>programming_site_query site \in queries.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
217	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
218	<code>op programming_site_reprograms</code>	Defines operation <code>programming_site_reprograms</code> , a total mathematical function or abbreviation used by later declarations.
219	<code>(programmed : programming_site list) (site : programming_site) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
220	<code>has (fun old => programming_sites_conflict site old) programmed.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
221	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
222	<code>op programming_site_fresh</code>	Defines operation <code>programming_site_fresh</code> , a total mathematical function or abbreviation used by later declarations.
223	<code>(queries : ro_query list) (programmed : programming_site list)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
224	<code>(site : programming_site) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
225	<code>! programming_site_prequeried queries site /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
226	<code>! programming_site_reprograms programmed site.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
227	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
228	<code>op programming_transcript_fresh</code>	Defines operation <code>programming_transcript_fresh</code> , a total mathematical function or abbreviation used by later declarations.
229	<code>(md : mode) (queries : ro_query list)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
230	<code>(programmed : programming_site list) (tr : transcript) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
231	<code>programming_site_fresh queries programmed</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
232	<code>(actual_signature_programming_site md tr) /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
233	<code>! min_entropy_failure md tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
234	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
235	<code>op rom_counted_programming_bad</code>	Defines operation <code>rom_counted_programming_bad</code> , a total mathematical function or abbreviation used by later declarations.
236	<code>(queries : ro_query list) (programmed : programming_site list)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
237	<code>(entropy_bad : bool) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
238	<code>entropy_bad /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
239	<code>has (fun site => programming_site_prequeried queries site) programmed /\</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
240	<code>has (fun site => programming_site_reprograms programmed site) programmed.</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
241	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
242	<code>op rom_counted_state_bad</code>	Defines operation <code>rom_counted_state_bad</code> , a total mathematical function or abbreviation used by later declarations.
243	<code>(queries : ro_query list) (programmed : programming_site list)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
244	<code>(bad_min_entropy : bool) : bool =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
245	<code>rom_counted_programming_bad queries programmed bad_min_entropy.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
246	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
247	<code>op counted_rom_programming_bound_obligation : bool =</code>	Defines predicate <code>counted_rom_programming_bound_obligation</code> , an event or well-formedness condition used in later implications and game proofs.
248	<code>0%r <= counted_rom_programming_loss_term.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
249	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
250	module CountedLazyROM(0 : Oracle) = {	Defines module CountedLazyROM, a probabilistic program or game used to create events and success probabilities.
251	var hash_queries : ro_query list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
252	var programmed_sites : programming_site list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
253	var signature_sites : programming_site list	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
254	var bad_prequery : bool	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
255	var bad_reprogram : bool	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
256	var bad_min_entropy : bool	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
257	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
258	proc init() : unit = {	Defines procedure init. Its body is a probabilistic program whose result becomes part of a game event.
259	0.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
260	hash_queries <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
261	programmed_sites <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
262	signature_sites <- [];	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
263	bad_prequery <- false;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
264	bad_reprogram <- false;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
265	bad_min_entropy <- false;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
266	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
267	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
268	proc get(q : ro_query) : ro_output = {	Defines procedure get. Its body is a probabilistic program whose result becomes part of a game event.
269	var y : ro_output;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
270	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
271	y <@ 0.get(q);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
272	hash_queries <- q :: hash_queries;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
273	return y;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
274	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
275	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
276	proc program(site : programming_site) : unit = {	Defines procedure program. Its body is a probabilistic program whose result becomes part of a game event.
277	bad_prequery <-	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
278	bad_prequery \/ programming_site_prequeried hash_queries site;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
279	bad_reprogram <-	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
280	bad_reprogram \/ programming_site_reprograms programmed_sites site;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
281	programmed_sites <- site :: programmed_sites;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
282	<code>0.set(programming_site_query site, programming_site_output site);</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
283	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
284	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
285	<code>proc observe_signature_site(site : programming_site) : unit = {</code>	Defines procedure <code>observe_signature_site</code> . Its body is a probabilistic program whose result becomes part of a game event.
286	<code>signature_sites <- site :: signature_sites;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
287	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
288	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
289	<code>proc observe_transcript(md : mode, tr : transcript) : unit = {</code>	Defines procedure <code>observe_transcript</code> . Its body is a probabilistic program whose result becomes part of a game event.
290	<code>var site : programming_site;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
291	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
292	<code>site <- actual_signature_programming_site md tr;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
293	<code>bad_min_entropy <- bad_min_entropy \/ min_entropy_failure md tr;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
294	<code>signature_sites <- site :: signature_sites;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
295	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
296	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
297	<code>proc bad() : bool = {</code>	Defines procedure <code>bad</code> . Its body is a probabilistic program whose result becomes part of a game event.
298	<code>return bad_prequery \/ bad_reprogram \/ bad_min_entropy;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
299	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
300	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
301	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
302	<code>module type SigningCoinSampler = {</code>	Defines module interface <code>SigningCoinSampler</code> . It specifies the procedures that later games or adversaries may call.
303	<code>proc sample() : random_coins</code>	Defines procedure <code>sample</code> . Its body is a probabilistic program whose result becomes part of a game event.
304	<code>}.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
305	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
306	<code>module DirectSigningCoinSampler : SigningCoinSampler = {</code>	Defines module <code>DirectSigningCoinSampler</code> , a probabilistic program or game used to create events and success probabilities.
307	<code>proc sample() : random_coins = {</code>	Defines procedure <code>sample</code> . Its body is a probabilistic program whose result becomes part of a game event.
308	<code>var coins : random_coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
309	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
310	<code>coins <\$ drandom_coins;</code>	Samples from a distribution. Mathematically this introduces a random variable with the named law.
311	<code>return coins;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
312	<code>}</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
313	<code>}..</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
314	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
315	section CountedLazyROMStateFacts.	Starts section CountedLazyROMStateFacts, a scoped region for related declarations and hypotheses.
316	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
317	declare module O <: Oracle {-CountedLazyROM}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
318	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
319	lemma counted_lazy_rom_init_clears :	States checked lemma counted_lazy_rom_init_clears. The following proof script must derive this proposition from prior definitions and lemmas.
320	hoare[CountedLazyROM(0).init :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
321	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
322	! CountedLazyROM.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
323	! CountedLazyROM.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
324	! CountedLazyROM.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
325	CountedLazyROM.programmed_sites = []].	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
326	proof.	Begins the proof script for the preceding proposition.
327	proc.	Enters procedure-body proof mode for a module procedure.
328	wp.	Applies weakest-precondition reasoning to a program fragment.
329	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
330	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
331	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
332	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
333	lemma counted_lazy_rom_init_clears_hash_queries :	States checked lemma counted_lazy_rom_init_clears_hash_queries. The following proof script must derive this proposition from prior definitions and lemmas.
334	hoare[CountedLazyROM(0).init :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
335	true ==> CountedLazyROM.hash_queries = []].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
336	proof.	Begins the proof script for the preceding proposition.
337	proc.	Enters procedure-body proof mode for a module procedure.
338	wp.	Applies weakest-precondition reasoning to a program fragment.
339	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
340	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
341	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
342	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
343	lemma counted_lazy_rom_get_preserves_clear :	States checked lemma counted_lazy_rom_get_preserves_clear. The following proof script must derive this proposition from prior definitions and lemmas.
344	hoare[CountedLazyROM(0).get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
345	! CountedLazyROM.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
346	! CountedLazyROM.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
347	! CountedLazyROM.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
348	CountedLazyROM.programmed_sites = [] ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
349	! CountedLazyROM.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
350	! CountedLazyROM.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
351	<code>! CountedLazyROM.bad_min_entropy /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
352	<code>CountedLazyROM.programmed_sites = []].</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
353	<code>proof.</code>	Begins the proof script for the preceding proposition.
354	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
355	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
356	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
357	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
358	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
359	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
360	<code>lemma counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count n :</code>	States checked lemma <code>counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count</code> . The following proof script must derive this proposition from prior definitions and lemmas.
361	<code>hoare[CountedLazyROM(0).get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
362	<code>! ro_query_is_challenge q /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
363	<code>challenge_query_log_count CountedLazyROM.hash_queries <= n ==></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
364	<code>challenge_query_log_count CountedLazyROM.hash_queries <= n].</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
365	<code>proof.</code>	Begins the proof script for the preceding proposition.
366	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
367	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
368	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
369	<code>by auto => />; smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
370	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
371	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
372	<code>lemma counted_lazy_rom_get_preserves_bad_clear :</code>	States checked lemma <code>counted_lazy_rom_get_preserves_bad_clear</code> . The following proof script must derive this proposition from prior definitions and lemmas.
373	<code>hoare[CountedLazyROM(0).get :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
374	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
375	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
376	<code>! CountedLazyROM.bad_min_entropy ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
377	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
378	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
379	<code>! CountedLazyROM.bad_min_entropy].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
380	<code>proof.</code>	Begins the proof script for the preceding proposition.
381	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
382	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
383	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.
384	<code>by auto.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
385	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
386	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
387	<code>lemma counted_lazy_rom_get_preserves_min_entropy_clear :</code>	States checked lemma <code>counted_lazy_rom_get_preserves_min_entropy_clear</code> . The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
388	hoare[CountedLazyROM(0).get :	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
389	! CountedLazyROM.bad_min_entropy ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
390	! CountedLazyROM.bad_min_entropy].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
391	proof.	Begins the proof script for the preceding proposition.
392	proc.	Enters procedure-body proof mode for a module procedure.
393	wp.	Applies weakest-precondition reasoning to a program fragment.
394	call (_, true).	Uses a procedure specification or adversary call rule inside a program logic proof.
395	by auto.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
396	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
397	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
398	lemma counted_lazy_rom_observe_transcript_preserves_clear :	States checked lemma
399	hoare[CountedLazyROM(0).observe_transcript :	counted_lazy_rom_observe_transcript_preserves_clear. The following proof script must derive this proposition from prior definitions and lemmas.
400	! min_entropy_failure md tr /\	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
401	! CountedLazyROM.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
402	! CountedLazyROM.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
403	! CountedLazyROM.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
404	CountedLazyROM.programmed_sites = [] ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
405	! CountedLazyROM.bad_prequery /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
406	! CountedLazyROM.bad_reprogram /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
407	! CountedLazyROM.bad_min_entropy /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
408	CountedLazyROM.programmed_sites = []].	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
409	proof.	Begins the proof script for the preceding proposition.
410	proc.	Enters procedure-body proof mode for a module procedure.
411	wp.	Applies weakest-precondition reasoning to a program fragment.
412	skip.	Discharges a program-logic step where no state-changing command remains.
413	move=> &hr [min_ok [preq_ok [reprog_ok [entropy_ok programmed_ok]]]].	Introduces universally quantified variables or hypotheses into the proof context.
414	by rewrite min_ok entropy_ok.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
415	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
416	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
417	lemma counted_lazy_rom_observe_transcript_preserves_min_entropy_clear :	States checked lemma
418	hoare[CountedLazyROM(0).observe_transcript :	counted_lazy_rom_observe_transcript_preserves_min_entropy_clear. The following proof script must derive this proposition from prior definitions and lemmas.
419	! min_entropy_failure md tr /\	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
420	! CountedLazyROM.bad_min_entropy ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
421	! CountedLazyROM.bad_min_entropy].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
422	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
423	proc.	Begins the proof script for the preceding proposition.
424	wp.	Enters procedure-body proof mode for a module procedure.
425	skip.	Applies weakest-precondition reasoning to a program fragment.
		Discharges a program-logic step where no state-changing command remains.

Line	EasyCrypt source	Mathematical reading
426	<code>move=> &hr [min_ok entropy_ok].</code>	Introduces universally quantified variables or hypotheses into the proof context.
427	<code>by rewrite min_ok entropy_ok.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
428	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
429	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
430	<code>lemma counted_lazy_rom_observe_transcript_preserves_bad_clear :</code>	States checked lemma
		<code>counted_lazy_rom_observe_transcript_preserves_bad_clear</code> . The following proof script must derive this proposition from prior definitions and lemmas.
431	<code>hoare[CountedLazyROM(0).observe_transcript :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
432	<code>! min_entropy_failure md tr /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
433	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
434	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
435	<code>! CountedLazyROM.bad_min_entropy ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
436	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
437	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
438	<code>! CountedLazyROM.bad_min_entropy].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
439	<code>proof.</code>	Begins the proof script for the preceding proposition.
440	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
441	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
442	<code>skip.</code>	Discharges a program-logic step where no state-changing command remains.
443	<code>move=> &hr [min_ok [preq_ok [reprog_ok entropy_ok]]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
444	<code>by rewrite min_ok entropy_ok.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
445	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
446	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
447	<code>lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh :</code>	States checked lemma
		<code>counted_lazy_rom_program_preserves_bad_clear_if_fresh</code> . The following proof script must derive this proposition from prior definitions and lemmas.
448	<code>hoare[CountedLazyROM(0).program :</code>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
449	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
450	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
451	<code>! CountedLazyROM.bad_min_entropy /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
452	<code>programming_site_fresh</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
453	<code>CountedLazyROM.hash_queries</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
454	<code>CountedLazyROM.programmed_sites site ==></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
455	<code>! CountedLazyROM.bad_prequery /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
456	<code>! CountedLazyROM.bad_reprogram /\</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
457	<code>! CountedLazyROM.bad_min_entropy].</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
458	<code>proof.</code>	Begins the proof script for the preceding proposition.
459	<code>proc.</code>	Enters procedure-body proof mode for a module procedure.
460	<code>wp.</code>	Applies weakest-precondition reasoning to a program fragment.
461	<code>call (_, true).</code>	Uses a procedure specification or adversary call rule inside a program logic proof.

Line	EasyCrypt source	Mathematical reading
462	by auto => />.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
463	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
464	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
465	lemma counted_lazy_rom_program_preserves_min_entropy_clear :	States checked lemma
466	hoare[CountedLazyROM(0).program :	counted_lazy_rom_program_preserves_min_entropy_clear. The following proof script must derive this proposition from prior definitions and lemmas.
467	! CountedLazyROM.bad_min_entropy ==>	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
468	! CountedLazyROM.bad_min_entropy].	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
469	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
470	proc.	Begins the proof script for the preceding proposition.
471	wp.	Enters procedure-body proof mode for a module procedure.
472	call (_, true).	Applies weakest-precondition reasoning to a program fragment.
473	by auto.	Uses a procedure specification or adversary call rule inside a program logic proof.
474	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
475	<i>blank</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
476	lemma counted_lazy_rom_bad_preserves_min_entropy_clear :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
477	hoare[CountedLazyROM(0).bad :	States checked lemma
478	! CountedLazyROM.bad_min_entropy ==>	counted_lazy_rom_bad_preserves_min_entropy_clear. The following proof script must derive this proposition from prior definitions and lemmas.
479	! CountedLazyROM.bad_min_entropy].	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
480	proof.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
481	proc.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
482	skip.	Begins the proof script for the preceding proposition.
483	by auto.	Enters procedure-body proof mode for a module procedure.
484	qed.	Discharges a program-logic step where no state-changing command remains.
485	<i>blank</i>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
486	lemma counted_lazy_rom_bad_false_when_clear :	Ends the proof; EasyCrypt has accepted all remaining obligations.
487	hoare[CountedLazyROM(0).bad :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
488	! CountedLazyROM.bad_prequery /\	States checked lemma counted_lazy_rom_bad_false_when_clear. The following proof script must derive this proposition from prior definitions and lemmas.
489	! CountedLazyROM.bad_reprogram /\	Starts or states a Hoare proof: every terminating run from a precondition must satisfy the postcondition.
490	! CountedLazyROM.bad_min_entropy ==>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
491	! res].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
492	proof.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
493	proc.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
494	skip.	Begins the proof script for the preceding proposition.
495	move=> &hr [preq_ok [reprog_ok entropy_ok]].	Enters procedure-body proof mode for a module procedure.
496	by rewrite preq_ok reprog_ok entropy_ok.	Discharges a program-logic step where no state-changing command remains.
497	qed.	Introduces universally quantified variables or hypotheses into the proof context.
498	<i>blank</i>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
499	end section CountedLazyROMStateFacts.	Ends the proof; EasyCrypt has accepted all remaining obligations.
500	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
501	module type CountedROMInterface = {	Defines module interface CountedROMInterface. It specifies the procedures that later games or adversaries may call.
502	proc get(q : ro_query) : ro_output	Defines procedure get. Its body is a probabilistic program whose result becomes part of a game event.
503	proc program(site : programming_site) : unit	Defines procedure program. Its body is a probabilistic program whose result becomes part of a game event.
504	proc observe_signature_site(site : programming_site) : unit	Defines procedure observe_signature_site. Its body is a probabilistic program whose result becomes part of a game event.
505	proc observe_transcript(md : mode, tr : transcript) : unit	Defines procedure observe_transcript. Its body is a probabilistic program whose result becomes part of a game event.
506	}. 507 blank	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
508	module type CountedROMClient(C : CountedROMInterface) = {	Defines module interface CountedROMClient. It specifies the procedures that later games or adversaries may call.
509	proc run() : unit {	Defines procedure run. Its body is a probabilistic program whose result becomes part of a game event.
510	C.get,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
511	C.program,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
512	C.observe_signature_site,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
513	C.observe_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
514	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
515	}. 516 blank	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
517	module CountedLazyROMBadGame(	Defines module CountedLazyROMBadGame, a probabilistic program or game used to create events and success probabilities.
518	O : Oracle,	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
519	A : CountedROMClient	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
520	) = {	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
521	module C = CountedLazyROM(O)	Defines module C, a probabilistic program or game used to create events and success probabilities.
522	module A = A(C)	Defines module A, a probabilistic program or game used to create events and success probabilities.
523	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
524	proc main() : bool = {	Defines procedure main. Its body is a probabilistic program whose result becomes part of a game event.
525	var bad : bool;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
526	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
527	C.init();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
528	A.run();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
529	bad <@ C.bad();	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
530	return bad;	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
531	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
532	}. 533 blank	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
534	module ProgramChallenge(0 : Oracle) = {	Defines module ProgramChallenge, a probabilistic program or game used to create events and success probabilities.
535	proc program(site : programming_site) : unit = {	Defines procedure program. Its body is a probabilistic program whose result becomes part of a game event.
536	0.set(programming_site_query site, programming_site_output site);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
537	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
538	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
539	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
540	module type TranscriptProgrammer(0 : Oracle) = {	Defines module interface TranscriptProgrammer. It specifies the procedures that later games or adversaries may call.
541	proc program(md : mode, tr : transcript, y : ro_output) : unit	Defines procedure program. Its body is a probabilistic program whose result becomes part of a game event.
542	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
543	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
544	module TranscriptProgrammerFromSite(0 : Oracle) = {	Defines module TranscriptProgrammerFromSite, a probabilistic program or game used to create events and success probabilities.
545	proc program(md : mode, tr : transcript, y : ro_output) : unit = {	Defines procedure program. Its body is a probabilistic program whose result becomes part of a game event.
546	ProgramChallenge(0).program(programming_site_of_transcript md tr y);	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
547	}	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
548	}.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
549	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
550	lemma programming_site_self_matches md tr y :	States checked lemma programming_site_self_matches. The following proof script must derive this proposition from prior definitions and lemmas.
551	transcript_challenge_matches tr y =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
552	programming_site_matches_transcript md tr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
553	(programming_site_of_transcript md tr y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
554	proof.	Begins the proof script for the preceding proposition.
555	by rewrite /programming_site_matches_transcript	Discharges the current goal in one step using the cited simplification, lemma, or automation.
556	/programming_site_of_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
557	/programming_site_query /programming_site_output.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
558	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
559	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
560	lemma programming_site_self_no_reprogramming_failure md tr y :	States checked lemma programming_site_self_no_reprogramming_failure. The following proof script must derive this proposition from prior definitions and lemmas.
561	transcript_challenge_matches tr y =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
562	! reprogramming_failure md tr (programming_site_of_transcript md tr y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
563	proof.	Begins the proof script for the preceding proposition.
564	move=> match_y.	Introduces universally quantified variables or hypotheses into the proof context.
565	rewrite /reprogramming_failure.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
566	by rewrite (programming_site_self_matches md tr y match_y).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
567	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
568	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
569	lemma fs_with_aborts_failureE md tr site :	States checked lemma fs_with_aborts_failureE. The following proof script must derive this proposition from prior definitions and lemmas.
570	fs_with_aborts_failure md tr site =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
571	(reprogramming_failure md tr site \/ min_entropy_failure md tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
572	proof. by rewrite /fs_with_aborts_failure. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
573	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
574	lemma programming_site_prequeried_cons qs site :	States checked lemma programming_site_prequeried_cons. The following proof script must derive this proposition from prior definitions and lemmas.
575	programming_site_prequeried (programming_site_query site :: qs) site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
576	proof. by rewrite /programming_site_prequeried. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
577	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
578	lemma programming_site_prequeried_cons_preserve q qs site :	States checked lemma programming_site_prequeried_cons_preserve. The following proof script must derive this proposition from prior definitions and lemmas.
579	programming_site_prequeried qs site =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
580	programming_site_prequeried (q :: qs) site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
581	proof. by rewrite /programming_site_prequeried /=; smt. qed.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
582	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
583	lemma programming_site_prequeried_output_irrelevant qs q y1 y2 :	States checked lemma programming_site_prequeried_output_irrelevant. The following proof script must derive this proposition from prior definitions and lemmas.
584	programming_site_prequeried qs (q, y1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
585	programming_site_prequeried qs (q, y2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
586	proof. by rewrite /programming_site_prequeried /programming_site_query. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
587	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
588	lemma programming_site_prequeried_witness qs q y :	States checked lemma programming_site_prequeried_witness. The following proof script must derive this proposition from prior definitions and lemmas.
589	q \in qs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
590	programming_site_prequeried qs (q, y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
591	proof. by rewrite /programming_site_prequeried /programming_site_query. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
592	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
593	lemma honest_signing_programming_site_queryE md sk m ctx coins :	States checked lemma honest_signing_programming_site_queryE. The following proof script must derive this proposition from prior definitions and lemmas.
594	honest_signing_programming_site_query md sk m ctx coins =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
595	challenge_hash_query md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
596	(commitment_highbits md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
597	(commitment_lowbits md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
598	(message_hash (public_key_of_secret md sk) ctx m).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
599	proof.	Begins the proof script for the preceding proposition.
600	by rewrite /honest_signing_programming_site_query	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
601	/honest_signing_programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
602	/actual_signature_programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
603	/signature_programming_site_of_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
604	/programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
605	/transcript_from_honest_signing	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
606	/transcript_of_signature	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
607	/sig_commitment_highbits /sig_commitment_lowbits	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
608	/sig_message_hash /sign_internal /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
609	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
610	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
611	op programmed_site_entropy_token_from_query (q : ro_query) : int =	Defines integer-valued parameter programmed_site_entropy_token_from_query, used for dimensions, budgets, support sizes, or bounds.
612	with q = ChallengeHashQuery x => nth 0 x.'3 0	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
613	with q = MessageHashQuery _ => 0	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
614	with q = MatrixExpandQuery _ => 0	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
615	with q = SamplerExpandQuery _ => 0.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
616	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
617	lemma honest_signing_programming_site_query_entropy_token	States checked lemma honest_signing_programming_site_query_entropy_token. The following proof script must derive this proposition from prior definitions and lemmas.
618	md sk m ctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
619	programmed_site_entropy_token_from_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
620	(honest_signing_programming_site_query md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
621	signing_entropy_token_of_coins coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
622	proof.	Begins the proof script for the preceding proposition.
623	rewrite honest_signing_programming_site_queryE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
624	/challenge_hash_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
625	/programmed_site_entropy_token_from_query /=.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
626	by apply commitment_lowbits_entropy_token.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
627	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
628	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
629	lemma honest_signing_programming_site_query_token_injective	States checked lemma honest_signing_programming_site_query_token_injective. The following proof script must derive this proposition from prior definitions and lemmas.
630	md sk m ctx x q :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
631	signing_entropy_token_in_range x =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
632	honest_signing_programming_site_query md sk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
633	(signing_entropy_token_to_coins x) = q =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
634	<code>x = programmed_site_entropy_token_from_query q.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
635	<code>proof.</code>	Begins the proof script for the preceding proposition.
636	<code>move=> xrange query_eq.</code>	Introduces universally quantified variables or hypotheses into the proof context.
637	<code>have h :=</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
638	<code>honest_signing_programming_site_query_entropy_token</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
639	<code>md sk m ctx (signing_entropy_token_to_coins x).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
640	<code>rewrite query_eq (signing_entropy_token_to_coins_tokenK x) in h.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
641	<code>by rewrite h.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
642	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
643	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
644	<code>lemma signing_distribution_programming_site_query_spread</code>	States checked lemma
645	<code>(d : random_coins distr) md sk m ctx q :</code>	<code>signing_distribution_programming_site_query_spread</code> . The following proof script must derive this proposition from prior definitions and lemmas.
646	<code>signing_randomness_token_spread d =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
647	<code>mu d</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
648	<code>(fun coins =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
649	<code>honest_signing_programming_site_query md sk m ctx coins = q) <=</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
650	<code>1%r / challenge_support_cardinality_lower_bound.</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
651	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
652	<code>move=> token_spread.</code>	Begins the proof script for the preceding proposition.
653	<code>apply (ler_trans</code>	Introduces universally quantified variables or hypotheses into the proof context.
654	<code>(mu d</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
655	<code>(fun coins =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
656	<code>signing_entropy_token_of_coins coins =</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
657	<code>programmed_site_entropy_token_from_query q)))</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
658	<code>+ apply mu_le => coins _ query_eq.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
659	<code>have h :=</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
660	<code>honest_signing_programming_site_query_entropy_token</code>	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
661	<code>md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
662	<code>rewrite query_eq in h.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
663	<code>by smt.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
664	<code>apply (ler_trans</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
665	<code>signing_randomness_point_bound).</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
666	<code>+ by rewrite /signing_randomness_token_spread in token_spread;</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
667	<code>apply token_spread.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
		Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
668	by rewrite /signing_randomness_point_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
669	/signing_entropy_token_cardinality	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
670	/challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
671	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
672	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
673	lemma honest_signing_programming_site_query_spread	States checked lemma honest_signing_programming_site_query_spread.
674	md sk m ctx q :	The following proof script must derive this proposition from prior definitions and lemmas.
675	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
676	(fun coins =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
677	honest_signing_programming_site_query md sk m ctx coins = q) <=	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
678	1%r / challenge_support_cardinality_lower_bound.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
679	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
680	by apply signing_distribution_programming_site_query_spread;	Begins the proof script for the preceding proposition.
681	apply signing_coin_distribution_token_spread.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
682	qed.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
683	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
684	lemma honest_signing_programming_site_query_spread_bound	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
685	md sk m ctx qs :	States checked lemma honest_signing_programming_site_query_spread_bound.
686	programmed_site_query_min_entropy_bound md sk m ctx qs	The following proof script must derive this proposition from prior definitions and lemmas.
687	(1%r / challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
688	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
689	rewrite /programmed_site_query_min_entropy_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
690	move=> q _.	Begins the proof script for the preceding proposition.
691	by apply honest_signing_programming_site_query_spread.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
692	qed.	Introduces universally quantified variables or hypotheses into the proof context.
693	blank	Discharges the current goal in one step using the cited simplification, lemma, or automation.
694	lemma honest_signing_programming_site_query_spread_bound_for	Ends the proof; EasyCrypt has accepted all remaining obligations.
695	(d : random_coins distr) md sk m ctx qs :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
696	signing_randomness_token_spread d =>	States checked lemma honest_signing_programming_site_query_spread_bound_for.
697	programmed_site_query_min_entropy_bound_for d md sk m ctx qs	The following proof script must derive this proposition from prior definitions and lemmas.
698	(1%r / challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
699	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
700	move=> spread q _.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
701	by apply signing_distribution_programming_site_query_spread.	Begins the proof script for the preceding proposition.
		Introduces universally quantified variables or hypotheses into the proof context.
		Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
702	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
703	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
704	lemma honest_signing_programming_site_query_is_challenge	States checked lemma honest_signing_programming_site_query_is_challenge. The following proof script must derive this proposition from prior definitions and lemmas.
705	md sk m ctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
706	ro_query_is_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
707	(honest_signing_programming_site_query md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
708	proof.	Begins the proof script for the preceding proposition.
709	by rewrite honest_signing_programming_site_queryE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
710	/challenge_hash_query /ro_query_is_challenge.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
711	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
712	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
713	lemma programming_site_prequeried_honest_challenge_filter	States checked lemma programming_site_prequeried_honest_challenge_filter. The following proof script must derive this proposition from prior definitions and lemmas.
714	qs md sk m ctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
715	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
716	(honest_signing_programming_site md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
717	programming_site_prequeried (filter ro_query_is_challenge qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
718	(honest_signing_programming_site md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
719	proof.	Begins the proof script for the preceding proposition.
720	rewrite /programming_site_prequeried	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
721	/honest_signing_programming_site_query.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
722	rewrite mem_filter.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
723	by rewrite honest_signing_programming_site_query_is_challenge.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
724	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
725	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
726	lemma programming_site_prequeried_honest_message_consE	States checked lemma programming_site_prequeried_honest_message_consE. The following proof script must derive this proposition from prior definitions and lemmas.
727	pk ctx m qs md sk sm sctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
728	programming_site_prequeried (message_hash_query pk ctx m :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
729	(honest_signing_programming_site md sk sm sctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
730	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
731	(honest_signing_programming_site md sk sm sctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
732	proof.	Begins the proof script for the preceding proposition.
733	rewrite /programming_site_prequeried	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
734	/programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
735	/honest_signing_programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
736	/actual_signature_programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
737	/signature_programming_site_of_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
738	/transcript_signature_challenge_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
739	/transcript_from_honest_signing	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
740	/transcript_of_signature	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
741	/message_hash_query /challenge_hash_query /=. .	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
742	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
743	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
744	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
745	lemma honest_signing_programming_site_outputE md sk m ctx coins :	States checked lemma honest_signing_programming_site_outputE. The following proof script must derive this proposition from prior definitions and lemmas.
746	programming_site_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
747	(honest_signing_programming_site md sk m ctx coins) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
748	ro_output_of_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
749	(challenge_hash md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
750	(commitment_highbits md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
751	(commitment_lowbits md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
752	(message_hash (public_key_of_secret md sk) ctx m)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
753	proof.	Begins the proof script for the preceding proposition.
754	by rewrite /honest_signing_programming_site	Discharges the current goal in one step using the cited simplification, lemma, or automation.
755	/actual_signature_programming_site	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
756	/signature_programming_site_of_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
757	/programming_site_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
758	/transcript_signature_challenge_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
759	/transcript_from_honest_signing	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
760	/transcript_of_signature /sig_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
761	/sign_internal /=. .	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
762	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
763	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
764	lemma honest_signing_programming_site_functional_output	States checked lemma honest_signing_programming_site_functional_output. The following proof script must derive this proposition from prior definitions and lemmas.
765	md sk1 m1 ctx1 coins1 sk2 m2 ctx2 coins2 :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
766	honest_signing_programming_site_query md sk1 m1 ctx1 coins1 =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
767	honest_signing_programming_site_query md sk2 m2 ctx2 coins2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
768	programming_site_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
769	(honest_signing_programming_site md sk1 m1 ctx1 coins1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
770	programming_site_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
771	(honest_signing_programming_site md sk2 m2 ctx2 coins2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
772	proof.	Begins the proof script for the preceding proposition.
773	rewrite !honest_signing_programming_site_queryE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
774	!honest_signing_programming_site_outputE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
775	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
776	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
777	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
778	lemma honest_signing_programming_sites_no_conflict	States checked lemma honest_signing_programming_sites_no_conflict. The following proof script must derive this proposition from prior definitions and lemmas.
779	md sk1 m1 ctx1 coins1 sk2 m2 ctx2 coins2 :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
780	! programming_sites_conflict	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
781	(honest_signing_programming_site md sk1 m1 ctx1 coins1)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
782	(honest_signing_programming_site md sk2 m2 ctx2 coins2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
783	proof.	Begins the proof script for the preceding proposition.
784	rewrite /programming_sites_conflict /programming_sites_same_query.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
785	case: (programming_site_query	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
786	(honest_signing_programming_site md sk1 m1 ctx1 coins1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
787	programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
788	(honest_signing_programming_site md sk2 m2 ctx2 coins2)) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
789	move=> query_eq.	Introduces universally quantified variables or hypotheses into the proof context.
790	rewrite /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
791	by apply (honest_signing_programming_site_functional_output	Discharges the current goal in one step using the cited simplification, lemma, or automation.
792	md sk1 m1 ctx1 coins1 sk2 m2 ctx2 coins2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
793	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
794	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
795	lemma actual_signature_programming_site_sign_internalE	States checked lemma actual_signature_programming_site_sign_internalE. The following proof script must derive this proposition from prior definitions and lemmas.
796	md pk sk m ctx coins :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
797	pk = public_key_of_secret md sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
798	actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
799	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
800	(sign_internal md sk m ctx coins)) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
801	honest_signing_programming_site md sk m ctx coins.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
802	proof.	Begins the proof script for the preceding proposition.
803	move=> pkE.	Introduces universally quantified variables or hypotheses into the proof context.
804	rewrite /honest_signing_programming_site /transcript_from_honest_signing.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
805	by rewrite pkE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
806	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
807	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
808	<code>lemma actual_signature_programming_site_sign_internal_any_pkE</code>	States checked lemma actual_signature_programming_site_sign_internal_any_pkE. The following proof script must derive this proposition from prior definitions and lemmas.
809	<code>md pk sk m ctx coins :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
810	<code>actual_signature_programming_site md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
811	<code>(transcript_of_signature md pk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
812	<code>(sign_internal md sk m ctx coins)) =</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
813	<code>honest_signing_programming_site md sk m ctx coins.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
814	<code>proof.</code>	Begins the proof script for the preceding proposition.
815	<code>by rewrite /honest_signing_programming_site /transcript_from_honest_signing</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
816	<code>/actual_signature_programming_site</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
817	<code>/signature_programming_site_of_transcript</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
818	<code>/transcript_signature_challenge_query</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
819	<code>/transcript_signature_challenge_output</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
820	<code>/transcript_of_signature /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
821	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
822	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
823	<code>lemma actual_signature_programming_site_prequeried_after_cons</code>	States checked lemma actual_signature_programming_site_prequeried_after_cons. The following proof script must derive this proposition from prior definitions and lemmas.
824	<code>md pk sk m ctx coins q qs :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
825	<code>pk = public_key_of_secret md sk =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
826	<code>programming_site_prequeried qs</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
827	<code>(honest_signing_programming_site md sk m ctx coins) =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
828	<code>programming_site_prequeried (q :: qs)</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
829	<code>(actual_signature_programming_site md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
830	<code>(transcript_of_signature md pk m ctx</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
831	<code>(sign_internal md sk m ctx coins)))</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
832	<code>proof.</code>	Begins the proof script for the preceding proposition.
833	<code>move=> pkE pre.</code>	Introduces universally quantified variables or hypotheses into the proof context.
834	<code>rewrite (actual_signature_programming_site_sign_internalE</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
835	<code>md pk sk m ctx coins pkE).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
836	<code>by apply programming_site_prequeried_cons_preserve.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
837	<i>qed.</i>	Ends the proof; EasyCrypt has accepted all remaining obligations.
838	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
839	lemma actual_signature_programming_site_prequeried_after_message_hash	States checked lemma actual_signature_programming_site_prequeried_after_message_hash. The following proof script must derive this proposition from prior definitions and lemmas.
840	md pk sk m ctx coins qpk qctx qm qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
841	pk = public_key_of_secret md sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
842	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
843	(honest_signing_programming_site md sk m ctx coins) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
844	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
845	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
846	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
847	(sign_internal md sk m ctx coins)))	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
848	proof.	Begins the proof script for the preceding proposition.
849	move=> pkE pre.	Introduces universally quantified variables or hypotheses into the proof context.
850	by apply (actual_signature_programming_site_prequeried_after_cons	Discharges the current goal in one step using the cited simplification, lemma, or automation.
851	md pk sk m ctx coins (message_hash_query qpk qctx qm) qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
852	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
853	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
854	lemma actual_signature_programming_site_prequeried_message_hashE	States checked lemma actual_signature_programming_site_prequeried_message_hashE. The following proof script must derive this proposition from prior definitions and lemmas.
855	md pk sk m ctx coins qpk qctx qm qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
856	pk = public_key_of_secret md sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
857	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
858	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
859	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
860	(sign_internal md sk m ctx coins))) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
861	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
862	(honest_signing_programming_site md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
863	proof.	Begins the proof script for the preceding proposition.
864	move=> pkE.	Introduces universally quantified variables or hypotheses into the proof context.
865	rewrite (actual_signature_programming_site_sign_internalE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
866	md pk sk m ctx coins pkE).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
867	by rewrite programming_site_prequeried_honest_message_consE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
868	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
869	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
870	lemma actual_signature_programming_site_prequeried_message_hash_any_pkE	States checked lemma actual_signature_programming_site_prequeried_message_hash_any_pkE. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
871	md pk sk m ctx coins qpk qctx qm qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
872	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
873	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
874	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
875	(sign_internal md sk m ctx coins))) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
876	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
877	(honest_signing_programming_site md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
878	proof.	Begins the proof script for the preceding proposition.
879	rewrite (actual_signature_programming_site_sign_internal_any_pkE	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
880	md pk sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
881	by rewrite programming_site_prequeried_honest_message_consE.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
882	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
883	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
884	op programming_site_conflict_free	Defines operation programming_site_conflict_free, a total mathematical function or abbreviation used by later declarations.
885	(site : programming_site) (sites : programming_site list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
886	all (fun old => ! programming_sites_conflict site old) sites.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
887	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
888	op programming_site_log_conflict_free_for_honest	Defines operation programming_site_log_conflict_free_for_honest, a total mathematical function or abbreviation used by later declarations.
889	(md : mode) (sk : skey) (sites : programming_site list) : bool =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
890	forall m ctx coins,	Part of a universal mathematical condition that must hold for all listed values.
891	programming_site_conflict_free	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
892	(honest_signing_programming_site md sk m ctx coins) sites.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
893	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
894	lemma programming_site_log_conflict_free_for_honest_nil md sk :	States checked lemma programming_site_log_conflict_free_for_honest_nil. The following proof script must derive this proposition from prior definitions and lemmas.
895	programming_site_log_conflict_free_for_honest md sk [].	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
896	proof. by rewrite /programming_site_log_conflict_free_for_honest	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
897	/programming_site_conflict_free.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
898	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
899	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
900	lemma programming_site_log_conflict_free_for_honest_cons	States checked lemma programming_site_log_conflict_free_for_honest_cons. The following proof script must derive this proposition from prior definitions and lemmas.
901	md sk m ctx coins sites :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
902	programming_site_log_conflict_free_for_honest md sk sites =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
903	programming_site_log_conflict_free_for_honest md sk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
904	(honest_signing_programming_site md sk m ctx coins :: sites).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
905	proof.	Begins the proof script for the preceding proposition.
906	rewrite /programming_site_log_conflict_free_for_honest.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
907	move=> log m' ctx' coins'.	Introduces universally quantified variables or hypotheses into the proof context.
908	rewrite /programming_site_conflict_free /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
909	split.	Splits a conjunction or structured goal into independent proof obligations.
910	+ by apply honest_signing_programming_sites_no_conflict.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
911	by apply log.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
912	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
913	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
914	lemma programming_site_conflict_free_no_reprograms site sites :	States checked lemma <code>programming_site_conflict_free_no_reprograms</code> . The following proof script must derive this proposition from prior definitions and lemmas.
915	programming_site_conflict_free site sites =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
916	! programming_site_reprograms sites site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
917	proof.	Begins the proof script for the preceding proposition.
918	rewrite /programming_site_conflict_free /programming_site_reprograms.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
919	move/allP=> no_conflict.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
920	apply/hasPn=> old old_mem.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
921	by apply no_conflict.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
922	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
923	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
924	lemma actual_signature_programming_site_sign_internal_conflict_step	States checked lemma <code>actual_signature_programming_site_sign_internal_conflict_step</code> . The following proof script must derive this proposition from prior definitions and lemmas.
925	md pk sk m ctx coins sites :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
926	pk = public_key_of_secret md sk =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
927	programming_site_log_conflict_free_for_honest md sk sites =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
928	! programming_site_reprograms sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
929	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
930	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
931	(sign_internal md sk m ctx coins))) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
932	programming_site_log_conflict_free_for_honest md sk	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
933	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
934	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
935	(sign_internal md sk m ctx coins)) :: sites).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
936	proof.	Begins the proof script for the preceding proposition.
937	move=> pkE log.	Introduces universally quantified variables or hypotheses into the proof context.
938	have siteE :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
939	actual_signature_programming_site_sign_internalE	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
940	md pk sk m ctx coins pkE.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
941	split.	Splits a conjunction or structured goal into independent proof obligations.
942	+ rewrite siteE.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
943	apply programming_site_conflict_free_no_reprograms.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
944	by apply log.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
945	rewrite siteE.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
946	by apply programming_site_log_conflict_free_for_honest_cons.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
947	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
948	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
949	lemma programmed_site_reprogram_one_step_conflict_free_zero	States checked lemma programmed_site_reprogram_one_step_conflict_free_zero. The following proof script must derive this proposition from prior definitions and lemmas.
950	md sk m ctx sites :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
951	(forall coins,	Part of a universal mathematical condition that must hold for all listed values.
952	programming_site_conflict_free	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
953	(honest_signing_programming_site md sk m ctx coins) sites) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
954	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
955	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
956	programming_site_reprograms sites	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
957	(honest_signing_programming_site md sk m ctx coins)) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
958	0%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
959	proof.	Begins the proof script for the preceding proposition.
960	move=> conflict_free.	Introduces universally quantified variables or hypotheses into the proof context.
961	rewrite (mu_eq _ _ (fun (_ : random_coins) => false)).	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
962	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
963	by rewrite (programming_site_conflict_free_no_reprograms	Discharges the current goal in one step using the cited simplification, lemma, or automation.
964	(honest_signing_programming_site md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
965	sites (conflict_free coins)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
966	by rewrite mu0.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
967	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
968	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
969	lemma programmed_site_prequery_one_step_fset_bound_for	States checked lemma programmed_site_prequery_one_step_fset_bound_for. The following proof script must derive this proposition from prior definitions and lemmas.
970	(d : random_coins distr) md sk m ctx qs bd :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
971	programmed_site_query_min_entropy_bound_for d md sk m ctx qs bd =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
972	mu d	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
973	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
974	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
975	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
976	(card (oflist qs))%r * bd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
977	proof.	Begins the proof script for the preceding proposition.
978	rewrite /programmed_site_query_min_entropy_bound_for	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
979	/programming_site_prequeried	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
980	/honest_signing_programming_site_query.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
981	move=> spread.	Introduces universally quantified variables or hypotheses into the proof context.
982	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
983	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
984	exists q, q \in oflist qs /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
985	programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
986	(honest_signing_programming_site md sk m ctx coins) = q)).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
987	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
988	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
989	apply (mu_mem_le_gen (oflist qs) d	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
990	(fun q coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
991	programming_site_query	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
992	(honest_signing_programming_site md sk m ctx coins) = q)	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
993	bd).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
994	move=> q q_mem.	Introduces universally quantified variables or hypotheses into the proof context.
995	apply spread.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
996	by rewrite -mem_oflist.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
997	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
998	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
999	lemma programmed_site_prequery_one_step_fset_bound	States checked lemma programmed_site_prequery_one_step_fset_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1000	md sk m ctx qs bd :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1001	programmed_site_query_min_entropy_bound md sk m ctx qs bd =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1002	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1003	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1004	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1005	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1006	(card (oflist qs))%r * bd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1007	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
1008	rewrite /programmed_site_query_min_entropy_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1009	by apply programmed_site_prequery_one_step_fset_bound_for.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1010	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1011	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1012	lemma programmed_site_prequery_one_step_budget_bound_for	States checked lemma
1013	(d : random_coins distr) md sk m ctx qs :	programmed_site_prequery_one_step_budget_bound_for. The following proof script must derive this proposition from prior definitions and lemmas.
1014	(card (oflist qs))%r <= rom_hash_query_budget + 1%r =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1015	programmed_site_query_min_entropy_bound_for d md sk m ctx qs	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1016	(1%r / challenge_support_cardinality_lower_bound) =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1017	mu d	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1018	(fun coins =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1019	programming_site_prequeried qs	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1020	(honest_signing_programming_site md sk m ctx coins)) <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1021	(rom_hash_query_budget + 1%r) /	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1022	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1023	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1024	move=> card_budget spread.	Begins the proof script for the preceding proposition.
1025	have h :=	Introduces universally quantified variables or hypotheses into the proof context.
1026	programmed_site_prequery_one_step_fset_bound_for	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1027	d md sk m ctx qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1028	(1%r / challenge_support_cardinality_lower_bound) spread.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1029	apply (ler_trans ((card (oflist qs))%r *	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1030	(1%r / challenge_support_cardinality_lower_bound))).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1031	+ by apply h.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1032	have inv_nonneg :	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1033	0%r <= inv challenge_support_cardinality_lower_bound.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1034	+ by rewrite invr_ge0; apply challenge_support_cardinality_lower_bound_nonnegative.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1035	have := ler_wpmul2r	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1036	(1%r / challenge_support_cardinality_lower_bound)	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1037	inv_nonneg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1038	(card (oflist qs))%r	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1039	(rom_hash_query_budget + 1%r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1040	card_budget.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1041	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1042	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1043	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1044	lemma programmed_site_prequery_one_step_budget_bound	States checked lemma programmed_site_prequery_one_step_budget_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1045	md sk m ctx qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1046	(card (oflist qs))%r <= rom_hash_query_budget + 1%r =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1047	programmed_site_query_min_entropy_bound md sk m ctx qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1048	(1%r / challenge_support_cardinality_lower_bound) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1049	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1050	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1051	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1052	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1053	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1054	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1055	proof.	Begins the proof script for the preceding proposition.
1056	move=> card_budget.	Introduces universally quantified variables or hypotheses into the proof context.
1057	rewrite /programmed_site_query_min_entropy_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1058	by apply programmed_site_prequery_one_step_budget_bound_for.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1059	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1060	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1061	lemma programmed_site_prequery_one_step_challenge_fset_bound_for	States checked lemma programmed_site_prequery_one_step_challenge_fset_bound_for. The following proof script must derive this proposition from prior definitions and lemmas.
1062	(d : random_coins distr) md sk m ctx qs bd :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1063	programmed_site_query_min_entropy_bound_for d md sk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1064	(filter ro_query_is_challenge qs) bd =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1065	mu d	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1066	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1067	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1068	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1069	(card (oflist (filter ro_query_is_challenge qs)))%r * bd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1070	proof.	Begins the proof script for the preceding proposition.
1071	move=> spread.	Introduces universally quantified variables or hypotheses into the proof context.
1072	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1073	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1074	programming_site_prequeried (filter ro_query_is_challenge qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1075	(honest_signing_programming_site md sk m ctx coins))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1076	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1077	by rewrite programming_site_prequeried_honest_challenge_filter.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1078	by apply programmed_site_prequery_one_step_fset_bound_for.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1079	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1080	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1081	lemma programmed_site_prequery_one_step_challenge_fset_bound	States checked lemma programmed_site_prequery_one_step_challenge_fset_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1082	md sk m ctx qs bd :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1083	programmed_site_query_min_entropy_bound md sk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1084	(filter ro_query_is_challenge qs) bd =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1085	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1086	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1087	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1088	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1089	(card (oflist (filter ro_query_is_challenge qs)))%r * bd.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1090	proof.	Begins the proof script for the preceding proposition.
1091	rewrite /programmed_site_query_min_entropy_bound.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1092	by apply programmed_site_prequery_one_step_challenge_fset_bound_for.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1093	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1094	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1095	lemma programmed_site_prequery_one_step_challenge_budget_bound_for	States checked lemma programmed_site_prequery_one_step_challenge_budget_bound_for. The following proof script must derive this proposition from prior definitions and lemmas.
1096	(d : random_coins distr) md sk m ctx qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1097	(card (oflist (filter ro_query_is_challenge qs)))%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1098	rom_hash_query_budget + 1%r =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1099	programmed_site_query_min_entropy_bound_for d md sk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1100	(filter ro_query_is_challenge qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1101	(1%r / challenge_support_cardinality_lower_bound) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1102	mu d	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1103	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1104	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1105	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1106	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1107	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1108	proof.	Begins the proof script for the preceding proposition.

Line	EasyCrypt source	Mathematical reading
1109	move=> card_budget spread.	Introduces universally quantified variables or hypotheses into the proof context.
1110	have h :=	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1111	programmed_site_prequery_one_step_challenge_fset_bound_for	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1112	d md sk m ctx qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1113	(1%r / challenge_support_cardinality_lower_bound) spread.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1114	apply (ler_trans ((card (oflist (filter ro_query_is_challenge qs)))%r *	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1115	(1%r / challenge_support_cardinality_lower_bound))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1116	+ by apply h.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1117	have inv_nonneg :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1118	0%r <= inv challenge_support_cardinality_lower_bound.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1119	+ by rewrite invr_ge0; apply challenge_support_cardinality_lower_bound_nonnegative.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1120	have := ler_wpmul2r	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1121	(1%r / challenge_support_cardinality_lower_bound)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1122	inv_nonneg	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1123	(card (oflist (filter ro_query_is_challenge qs)))%r	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1124	(rom_hash_query_budget + 1%r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1125	card_budget.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1126	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1127	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1128	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1129	lemma programmed_site_prequery_one_step_challenge_budget_bound	States checked lemma programmed_site_prequery_one_step_challenge_budget_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1130	md sk m ctx qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1131	(card (oflist (filter ro_query_is_challenge qs)))%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1132	rom_hash_query_budget + 1%r =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1133	programmed_site_query_min_entropy_bound md sk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1134	(filter ro_query_is_challenge qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1135	(1%r / challenge_support_cardinality_lower_bound) =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1136	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1137	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1138	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1139	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1140	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1141	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1142	<code>proof.</code>	Begins the proof script for the preceding proposition.
1143	<code>move=> card_budget.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1144	<code>rewrite /programmed_site_query_min_entropy_bound.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1145	<code>by apply programmed_site_prequery_one_step_challenge_budget_bound_for.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1146	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1147	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1148	<code>lemma programmed_site_prequery_one_step_challenge_concrete_bound</code>	States checked lemma <code>programmed_site_prequery_one_step_challenge_concrete_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1149	<code>md sk m ctx qs :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1150	<code>(card (oflist (filter ro_query_is_challenge qs)))%r <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1151	<code>rom_hash_query_budget + 1%r =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1152	<code>mu drandom_coins</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1153	<code>(fun coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1154	<code>programming_site_prequeried qs</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1155	<code>(honest_signing_programming_site md sk m ctx coins)) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1156	<code>(rom_hash_query_budget + 1%r) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1157	<code>challenge_support_cardinality_lower_bound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1158	<code>proof.</code>	Begins the proof script for the preceding proposition.
1159	<code>move=> card_budget.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1160	<code>apply programmed_site_prequery_one_step_challenge_budget_bound.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1161	<code>+ by apply card_budget.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1162	<code>by apply honest_signing_programming_site_query_spread_bound.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1163	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1164	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1165	<code>lemma programmed_site_prequery_one_step_challenge_concrete_bound_for</code>	States checked lemma <code>programmed_site_prequery_one_step_challenge_concrete_bound_for</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1166	<code>(d : random_coins distr) md sk m ctx qs :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1167	<code>signing_randomness_token_spread d =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1168	<code>(card (oflist (filter ro_query_is_challenge qs)))%r <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1169	<code>rom_hash_query_budget + 1%r =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1170	<code>mu d</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1171	<code>(fun coins =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1172	<code>programming_site_prequeried qs</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1173	<code>(honest_signing_programming_site md sk m ctx coins)) <=</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1174	<code>(rom_hash_query_budget + 1%r) /</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1175	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1176	proof.	Begins the proof script for the preceding proposition.
1177	move=> spread card_budget.	Introduces universally quantified variables or hypotheses into the proof context.
1178	apply programmed_site_prequery_one_step_challenge_budget_bound_for.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1179	+ by apply card_budget.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1180	by apply honest_signing_programming_site_query_spread_bound_for.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1181	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1182	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1183	lemma challenge_query_log_count_budget_card_bound hc qs :	States checked lemma challenge_query_log_count_budget_card_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1184	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1185	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1186	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1187	(card (oflist (filter ro_query_is_challenge qs)))%r <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1188	rom_hash_query_budget + 1)%r.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1189	proof.	Begins the proof script for the preceding proposition.
1190	rewrite /challenge_query_log_count	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1191	/rom_hash_query_budget /hash_query_count /hash_query_budget_count.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1192	smt.	Calls SMT automation for first-order arithmetic, list, or algebraic side conditions.
1193	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1194	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1195	lemma programmed_site_prequery_one_step_challenge_counter_bound	States checked lemma programmed_site_prequery_one_step_challenge_counter_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1196	md sk m ctx qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1197	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1198	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1199	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1200	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1201	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1202	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1203	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1204	(rom_hash_query_budget + 1)%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1205	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1206	proof.	Begins the proof script for the preceding proposition.
1207	move=> hc_ge0 hc_budget query_count.	Introduces universally quantified variables or hypotheses into the proof context.
1208	apply programmed_site_prequery_one_step_challenge_concrete_bound.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
1209	by apply (challenge_query_log_count_budget_card_bound hc qs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1210	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1211	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1212	lemma programmed_site_prequery_one_step_challenge_counter_bound_for	States checked lemma programmed_site_prequery_one_step_challenge_counter_bound_for. The following proof script must derive this proposition from prior definitions and lemmas.
1213	(d : random_coins distr) md sk m ctx qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1214	signing_randomness_token_spread d =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1215	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1216	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1217	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1218	mu d	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1219	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1220	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1221	(honest_signing_programming_site md sk m ctx coins)) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1222	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1223	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1224	proof.	Begins the proof script for the preceding proposition.
1225	move=> spread hc_ge0 hc_budget query_count.	Introduces universally quantified variables or hypotheses into the proof context.
1226	apply programmed_site_prequery_one_step_challenge_concrete_bound_for.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1227	+ by apply spread.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1228	by apply (challenge_query_log_count_budget_card_bound hc qs).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1229	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1230	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1231	lemma programmed_site_prequery_one_step_message_hash_counter_bound	States checked lemma programmed_site_prequery_one_step_message_hash_counter_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1232	md pk sk m ctx qpk qctx qm qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1233	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1234	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1235	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1236	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1237	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1238	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1239	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1240	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1241	(sign_internal md sk m ctx coins)))) <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.

Line	EasyCrypt source	Mathematical reading
1242	(rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1243	challenge_support_cardinality_lower_bound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1244	proof.	Begins the proof script for the preceding proposition.
1245	move=> hc_ge0 hc_budget query_count.	Introduces universally quantified variables or hypotheses into the proof context.
1246	rewrite (mu_eq _ _	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1247	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1248	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1249	(honest_signing_programming_site md sk m ctx coins))).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1250	+ move=> coins.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1251	by rewrite (actual_signature_programming_site_prequeried_message_hash_any_pkE	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1252	md pk sk m ctx coins qpk qctx qm qs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1253	by apply (programmed_site_prequery_one_step_challenge_counter_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1254	md sk m ctx qs hc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1255	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1256	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1257	lemma direct_signing_coin_sampler_prequery_bound	States checked lemma direct_signing_coin_sampler_prequery_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1258	md sk m ctx qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1259	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1260	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1261	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1262	phoare[DirectSigningCoinSampler.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
1263	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1264	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1265	(honest_signing_programming_site md sk m ctx res)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1266	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1267	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1268	proof.	Begins the proof script for the preceding proposition.
1269	move=> hc_ge0 hc_budget query_count.	Introduces universally quantified variables or hypotheses into the proof context.
1270	bypr=> &m _.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1271	byphoare (_: true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1272	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1273	(honest_signing_programming_site md sk m ctx res)) => //.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1274	proc.	Enters procedure-body proof mode for a module procedure.
1275	rnd (fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1276	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1277	(honest_signing_programming_site md sk m ctx coins)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1278	auto => />.	Uses EasyCrypt automation for routine logical, arithmetic, or definitional obligations.
1279	by apply (programmed_site_prequery_one_step_challenge_counter_bound	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1280	md sk m ctx qs hc).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1281	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1282	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1283	lemma direct_signing_coin_sampler_budgeted_prequery_bound	States checked lemma
		direct_signing_coin_sampler_budgeted_prequery_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1284	md sk m ctx qs :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1285	challenge_query_log_count qs <= hash_query_budget_count + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1286	phoare[DirectSigningCoinSampler.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
1287	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1288	programming_site_prequeried qs	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1289	(honest_signing_programming_site md sk m ctx res)] <=	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1290	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1291	challenge_support_cardinality_lower_bound).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1292	proof.	Begins the proof script for the preceding proposition.
1293	move=> query_count.	Introduces universally quantified variables or hypotheses into the proof context.
1294	apply (direct_signing_coin_sampler_prequery_bound	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1295	md sk m ctx qs hash_query_budget_count).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1296	+ by rewrite /hash_query_budget_count; smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1297	+ by smt.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1298	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1299	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1300	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1301	lemma direct_signing_coin_sampler_message_hash_prequery_bound	States checked lemma
		direct_signing_coin_sampler_message_hash_prequery_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1302	md pk sk m ctx qpk qctx qm qs hc :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1303	0 <= hc =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1304	hc <= hash_query_budget_count =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1305	challenge_query_log_count qs <= hc + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1306	phoare[DirectSigningCoinSampler.sample :	Starts or states a probabilistic Hoare proof with an explicit probability bound.
1307	true ==>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1308	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1309	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1310	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with
1311	(sign_internal md sk m ctx res)))] <=	surrounding lines as part of the same mathematical object. Part of an inequality, usually comparing probabilities or bounding a concrete
1312	((rom_hash_query_budget + 1%r) /	loss term. Continuation line in the current declaration or proof. Read it with
1313	challenge_support_cardinality_lower_bound).	surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with
1314	proof.	surrounding lines as part of the same mathematical object.
1315	move=> hc_ge0 hc_budget query_count.	Begins the proof script for the preceding proposition.
1316	bypr=> &m0 _.	Introduces universally quantified variables or hypotheses into the proof
1317	byphoare (_, true ==>	context. Introduces implication structure: later proof steps may assume the
1318	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	antecedent to prove the conclusion.
1319	(actual_signature_programming_site md	Introduces implication structure: later proof steps may assume the
1320	(transcript_of_signature md pk m ctx	antecedent to prove the conclusion.
1321	(sign_internal md sk m ctx res)))] => //.	Continuation line in the current declaration or proof. Read it with
1322	proc.	surrounding lines as part of the same mathematical object.
1323	rnd (fun coins =>	Continuation line in the current declaration or proof. Read it with
1324	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	surrounding lines as part of the same mathematical object.
1325	(actual_signature_programming_site md	Continuation line in the current declaration or proof. Read it with
1326	(transcript_of_signature md pk m ctx	surrounding lines as part of the same mathematical object.
1327	(sign_internal md sk m ctx coins)))).	Continuation line in the current declaration or proof. Read it with
1328	auto => />.	surrounding lines as part of the same mathematical object.
1329	by apply (programmed_site_prequery_one_step_message_hash_counter_bound	Introduces implication structure: later proof steps may assume the
1330	md pk sk m ctx qpk qctx qm qs hc).	antecedent to prove the conclusion.
1331	qed.	Continuation line in the current declaration or proof. Read it with
1332	blank	surrounding lines as part of the same mathematical object.
1333	lemma direct_signing_coin_sampler_message_hash_budgeted_prequery_bound	Continuation line in the current declaration or proof. Read it with
1334	md pk sk m ctx qpk qctx qm qs :	surrounding lines as part of the same mathematical object.
1335	challenge_query_log_count qs <= hash_query_budget_count + 1 =>	Part of an inequality, usually comparing probabilities or bounding a concrete
1336	phoare[DirectSigningCoinSampler.sample :	loss term. Starts or states a probabilistic Hoare proof with an explicit probability
1337	true ==>	bound.
1338	programming_site_prequeried (message_hash_query qpk qctx qm :: qs)	Introduces implication structure: later proof steps may assume the
1339	(actual_signature_programming_site md	antecedent to prove the conclusion.
1340	(transcript_of_signature md pk m ctx	Continuation line in the current declaration or proof. Read it with
1341	(sign_internal md sk m ctx res)))] <=	surrounding lines as part of the same mathematical object.
1342	((rom_hash_query_budget + 1%r) /	Continuation line in the current declaration or proof. Read it with
1343	challenge_support_cardinality_lower_bound).	surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with

Line	EasyCrypt source	Mathematical reading
1344	<code>proof.</code>	Begins the proof script for the preceding proposition.
1345	<code>move=> query_count.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1346	<code>apply (direct_signing_coin_sampler_message_hash_prequery_bound</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1347	<code>md pk sk m ctx qpk qctx qm qs hash_query_budget_count).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1348	<code>+ by rewrite /hash_query_budget_count; smt.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1349	<code>+ by smt.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1350	<code>by smt.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1351	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1352	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1353	<code>lemma programmed_site_prequery_one_step_concrete_bound</code>	States checked lemma
1354	<code>md sk m ctx qs :</code>	<code>programmed_site_prequery_one_step_concrete_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1355	<code>(card (oflist qs))%r <= rom_hash_query_budget + 1%r =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1356	<code>mu drandom_coins</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1357	<code>(fun coins =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1358	<code>programming_site_prequeried qs</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1359	<code>(honest_signing_programming_site md sk m ctx coins)) <=</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1360	<code>(rom_hash_query_budget + 1%r) /</code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1361	<code>challenge_support_cardinality_lower_bound.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1362	<code>proof.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1363	<code>move=> card_budget.</code>	Begins the proof script for the preceding proposition.
1364	<code>apply programmed_site_prequery_one_step_budget_bound.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1365	<code>+ by apply card_budget.</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1366	<code>by apply honest_signing_programming_site_query_spread_bound.</code>	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1367	<code>qed.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1368	<code>blank</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1369	<code>lemma programmed_site_prequery_reprogram_one_step_concrete_bound</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1370	<code>md sk m ctx qs sites :</code>	States checked lemma
1371	<code>(forall coins,</code>	<code>programmed_site_prequery_reprogram_one_step_concrete_bound</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1372	<code>programming_site_conflict_free</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1373	<code>(honest_signing_programming_site md sk m ctx coins) sites) =></code>	Part of a universal mathematical condition that must hold for all listed values.
1374	<code>(card (oflist qs))%r <= rom_hash_query_budget + 1%r =></code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1375	<code>mu drandom_coins</code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1376	<code>(fun coins =></code>	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1377	<code>programming_site_prequeried qs</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1378	(honest_signing_programming_site md sk m ctx coins) \/ 1379 programming_site_reprograms sites 1380 (honest_signing_programming_site md sk m ctx coins)) <= 1381 (rom_hash_query_budget + 1%r) / 1382 challenge_support_cardinality_lower_bound. 1383 proof. 1384 move=> conflict_free card_budget. 1385 apply (ler_trans 1386 (mu drandom_coins 1387 (fun coins => 1388 programming_site_prequeried qs 1389 (honest_signing_programming_site md sk m ctx coins)))). 1390 + apply mu_le => coins _. 1391 case. 1392 + by move=> preq. 1393 move=> reprog. 1394 have no_reprog : 1395 ! programming_site_reprograms sites 1396 (honest_signing_programming_site md sk m ctx coins). 1397 + by apply (programming_site_conflict_free_no_reprograms 1398 (honest_signing_programming_site md sk m ctx coins) 1399 sites (conflict_free coins)). 1400 by move: no_reprog; rewrite reprog. 1401 by apply programmed_site_prequery_one_step_concrete_bound. 1402 qed. 1403 blank 1404 lemma programmed_site_query_min_entropy_bound_constant_impossible 1405 md sk m ctx qs bd q : 1406 q \in qs => 1407 (forall coins, 1408 honest_signing_programming_site_query md sk m ctx coins = q) => 1409 bd < 1%r => 1410 programmed_site_query_min_entropy_bound md sk m ctx qs bd => false.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Part of an inequality, usually comparing probabilities or bounding a concrete loss term. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Begins the proof script for the preceding proposition. Introduces universally quantified variables or hypotheses into the proof context. Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Solves a proof branch produced by a previous split, transitivity, or case analysis. Performs case analysis or induction, splitting the mathematical proof by constructors or list structure. Solves a proof branch produced by a previous split, transitivity, or case analysis. Introduces universally quantified variables or hypotheses into the proof context. Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Solves a proof branch produced by a previous split, transitivity, or case analysis. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Discharges the current goal in one step using the cited simplification, lemma, or automation. Discharges the current goal in one step using the cited simplification, lemma, or automation. Ends the proof; EasyCrypt has accepted all remaining obligations. Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases. States checked lemma programmed_site_query_min_entropy_bound_constant_impossible. The following proof script must derive this proposition from prior definitions and lemmas. Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion. Part of a universal mathematical condition that must hold for all listed values. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion. Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
1411	proof.	Begins the proof script for the preceding proposition.
1412	move=> q_mem query_constant bd_lt1 spread.	Introduces universally quantified variables or hypotheses into the proof context.
1413	have hit1 :	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1414	1%r =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1415	mu drandom_coins	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1416	(fun coins =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1417	honest_signing_programming_site_query md sk m ctx coins = q).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1418	+ rewrite eq_sym.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1419	apply eq1_mu.	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1420	+ by apply signing_coin_distribution_lossless.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1421	by move=> coins _; apply query_constant.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1422	have hit_le_bd := spread q q_mem.	Creates an intermediate claim. This mirrors a mathematical proof that first proves a sub-inequality or invariant.
1423	by smt.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1424	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1425	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1426	lemma programming_site_reprograms_conflict_cons site old programmed :	States checked lemma programming_site_reprograms_conflict_cons. The following proof script must derive this proposition from prior definitions and lemmas.
1427	programming_sites_conflict site old =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1428	programming_site_reprograms (old :: programmed) site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1429	proof.	Begins the proof script for the preceding proposition.
1430	move=> conflict.	Introduces universally quantified variables or hypotheses into the proof context.
1431	by rewrite /programming_site_reprograms /= conflict.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1432	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1433	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1434	lemma rom_counted_programming_bad_min_entropy queries programmed :	States checked lemma rom_counted_programming_bad_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
1435	rom_counted_programming_bad queries programmed true.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1436	proof. by rewrite /rom_counted_programming_bad. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1437	<i>blank</i>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1438	lemma rom_counted_programming_bad_prequery_cons queries programmed site	States checked lemma rom_counted_programming_bad_prequery_cons. The following proof script must derive this proposition from prior definitions and lemmas.
1439	entropy_bad :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1440	programming_site_prequeried queries site =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1441	rom_counted_programming_bad queries (site :: programmed) entropy_bad.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1442	proof.	Begins the proof script for the preceding proposition.
1443	move=> preq.	Introduces universally quantified variables or hypotheses into the proof context.
1444	rewrite /rom_counted_programming_bad /= preq.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.

Line	EasyCrypt source	Mathematical reading
1445	by case entropy_bad.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1446	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1447	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1448	lemma counted_rom_programming_bound_obligation_holds :	States checked lemma
1449	counted_rom_programming_bound_obligation.	counted_rom_programming_bound_obligation_holds. The following proof script must derive this proposition from prior definitions and lemmas.
1450	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1451	by rewrite /counted_rom_programming_bound_obligation	Begins the proof script for the preceding proposition.
1452	counted_rom_programming_loss_term_nonnegative.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1453	qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1454	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1455	section CountedLazyROMBadFlags.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1456	blank	Starts section CountedLazyROMBadFlags, a scoped region for related declarations and hypotheses.
1457	declare module O <: Oracle.	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1458	declare module A <: CountedROMClient {-O, -CountedLazyROM, -CountedLazyROMBadGame}.	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
1459	blank	Declares an abstract module parameter. Mathematically this quantifies over an oracle, adversary, or reduction satisfying the stated interface.
1460	op counted_lazy_rom_bad_flags_budget_sound (p : real) : bool =	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1461	p <= counted_rom_programming_loss_term.	Defines predicate counted_lazy_rom_bad_flags_budget_sound, an event or well-formedness condition used in later implications and game proofs.
1462	blank	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1463	lemma counted_lazy_rom_bad_flags_universal_bound &m :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1464	1%r >= Pr[CountedLazyROMBadGame(O, A).main() @ &m : res].	States checked lemma counted_lazy_rom_bad_flags_universal_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1465	proof.	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1466	byphoare (_: true ==> true) => //.	Begins the proof script for the preceding proposition.
1467	qed.	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1468	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1469	lemma counted_lazy_rom_bad_flags_query_budget_bound &m :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1470	counted_lazy_rom_bad_flags_budget_sound	States checked lemma
1471	(Pr[CountedLazyROMBadGame(O, A).main() @ &m : res]) =>	counted_lazy_rom_bad_flags_query_budget_bound. The following proof script must derive this proposition from prior definitions and lemmas.
1472	Pr[CountedLazyROMBadGame(O, A).main() @ &m : res] <=	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1473	counted_rom_programming_loss_term.	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1474	proof.	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1475	by rewrite /counted_lazy_rom_bad_flags_budget_sound.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1476	qed.	Begins the proof script for the preceding proposition.
1477	blank	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1478	lemma counted_lazy_rom_bad_flags_query_budget_bound_subunit &m :	Ends the proof; EasyCrypt has accepted all remaining obligations.
		Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
		States checked lemma
		counted_lazy_rom_bad_flags_query_budget_bound_subunit. The following proof script must derive this proposition from prior definitions and lemmas.

Line	EasyCrypt source	Mathematical reading
1479	counted_lazy_rom_bad_flags_budget_sound	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1480	(Pr[CountedLazyROMBadGame(0, A).main() @ &m : res]) =>	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1481	Pr[CountedLazyROMBadGame(0, A).main() @ &m : res] < 1/r.	Refers to a probability of an event in an EasyCrypt game; this is the central object of the security bound.
1482	proof.	Begins the proof script for the preceding proposition.
1483	move=> budget_sound.	Introduces universally quantified variables or hypotheses into the proof context.
1484	apply (1er_lt_trans counted_rom_programming_loss_term).	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.
1485	+ by move: budget_sound; rewrite /counted_lazy_rom_bad_flags_budget_sound.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1486	by apply counted_rom_programming_loss_term_subunit.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1487	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1488	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1489	end section CountedLazyROMBadFlags.	Closes the current section, discharging local module and variable scope.
1490	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1491	lemma fs_with_aborts_no_programming_failure md tr y :	States checked lemma fs_with_aborts_no_programming_failure. The following proof script must derive this proposition from prior definitions and lemmas.
1492	transcript_challenge_matches tr y =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1493	fs_with_aborts_failure md tr (programming_site_of_transcript md tr y) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1494	min_entropy_failure md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1495	proof.	Begins the proof script for the preceding proposition.
1496	move=> match_y.	Introduces universally quantified variables or hypotheses into the proof context.
1497	rewrite /fs_with_aborts_failure.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1498	by rewrite (programming_site_self_no_reprogramming_failure md tr y match_y).	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1499	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1500	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1501	lemma transcript_signature_challenge_output_matches tr :	States checked lemma transcript_signature_challenge_output_matches. The following proof script must derive this proposition from prior definitions and lemmas.
1502	transcript_signature_challenge_matches tr	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1503	(transcript_signature_challenge_output tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1504	proof.	Begins the proof script for the preceding proposition.
1505	by rewrite /transcript_signature_challenge_matches	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1506	/transcript_signature_challenge_output	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1507	/ro_challenge_hash /ro_output_to_challenge	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1508	/ro_output_of_challenge.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1509	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1510	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1511	lemma honest_transcript_challenge_entropy_support md sk m ctx coins :	States checked lemma honest_transcript_challenge_entropy_support. The following proof script must derive this proposition from prior definitions and lemmas.
1512	challenge_entropy_support_ok md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1513	(transcript_from_honest_signing md sk m ctx coins).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1514	<code>proof.</code>	Begins the proof script for the preceding proposition.
1515	<code>rewrite /challenge_entropy_support_ok /transcript_from_honest_signing</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1516	<code>/transcript_of_signature /transcript_challenge /signature_challenge</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1517	<code>/sig_challenge /sign_internal /challenge_hash /.=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1518	<code>by apply challenge_from_seed_sparse.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1519	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1520	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1521	<code>lemma honest_transcript_no_min_entropy_failure md sk m ctx coins :</code>	States checked lemma <code>honest_transcript_no_min_entropy_failure</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1522	<code>! min_entropy_failure md</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1523	<code>(transcript_from_honest_signing md sk m ctx coins).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1524	<code>proof.</code>	Begins the proof script for the preceding proposition.
1525	<code>by rewrite /min_entropy_failure</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1526	<code>honest_transcript_challenge_entropy_support.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1527	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1528	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1529	<code>lemma signing_transcript_relation_challenge_entropy_support</code>	States checked lemma <code>signing_transcript_relation_challenge_entropy_support</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1530	<code>md pk m ctx sig tr :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1531	<code>signing_transcript_relation md pk m ctx sig tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1532	<code>challenge_entropy_support_ok md tr.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1533	<code>proof.</code>	Begins the proof script for the preceding proposition.
1534	<code>rewrite /signing_transcript_relation /signing_simulation_relation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1535	<code>move=> rel.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1536	<code>case: rel => hsim rest.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1537	<code>case: hsim => _ witness.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1538	<code>case: rest => trE _.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1539	<code>elim witness => sk coins attempt.</code>	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1540	<code>rewrite trE /challenge_entropy_support_ok /transcript_of_signature</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1541	<code>/transcript_challenge /signature_challenge /.=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1542	<code>move: attempt.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1543	<code>rewrite /signing_attempt_relation.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1544	<code>move=> [_ [_ ->]].</code>	Introduces universally quantified variables or hypotheses into the proof context.
1545	<code>rewrite /sig_challenge /sign_internal /challenge_hash /.=.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1546	<code>by apply challenge_from_seed_sparse.</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1547	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1548	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1549	lemma signing_transcript_relation_no_min_entropy_failure	States checked lemma signing_transcript_relation_no_min_entropy_failure. The following proof script must derive this proposition from prior definitions and lemmas.
1550	md pk m ctx sig tr :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1551	signing_transcript_relation md pk m ctx sig tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1552	! min_entropy_failure md tr.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1553	proof.	Begins the proof script for the preceding proposition.
1554	move=> rel.	Introduces universally quantified variables or hypotheses into the proof context.
1555	rewrite /min_entropy_failure.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1556	by rewrite (signing_transcript_relation_challenge_entropy_support	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1557	md pk m ctx sig tr rel).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1558	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1559	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1560	lemma signing_record_relation_no_min_entropy_failure md pk r :	States checked lemma signing_record_relation_no_min_entropy_failure. The following proof script must derive this proposition from prior definitions and lemmas.
1561	signing_record_relation md pk r =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1562	! min_entropy_failure md (signing_record_transcript r).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1563	proof.	Begins the proof script for the preceding proposition.
1564	rewrite /signing_record_relation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1565	move=> rel.	Introduces universally quantified variables or hypotheses into the proof context.
1566	by apply (signing_transcript_relation_no_min_entropy_failure md pk	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1567	(signing_record_message r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1568	(signing_record_context r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1569	(signing_record_signature r)).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1570	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1571	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1572	lemma signing_record_log_sound_no_min_entropy md pk rs :	States checked lemma signing_record_log_sound_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
1573	signing_record_log_sound md pk rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1574	signing_record_log_min_entropy_clear md rs.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1575	proof.	Begins the proof script for the preceding proposition.
1576	rewrite /signing_record_log_sound /signing_record_log_min_entropy_clear.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1577	elim: rs => [r rs ih] //=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1578	move=> [rel sound].	Introduces universally quantified variables or hypotheses into the proof context.
1579	split.	Splits a conjunction or structured goal into independent proof obligations.
1580	+ by apply (signing_record_relation_no_min_entropy_failure md pk r).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1581	+ by apply ih.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1582	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1583	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.

Line	EasyCrypt source	Mathematical reading
1584	lemma signing_record_log_sound_transcripts_no_min_entropy md pk rs :	States checked lemma signing_record_log_sound_transcripts_no_min_entropy. The following proof script must derive this proposition from prior definitions and lemmas.
1585	signing_record_log_sound md pk rs =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1586	transcript_log_min_entropy_clear md (signing_record_transcripts rs).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1587	proof.	Begins the proof script for the preceding proposition.
1588	rewrite /signing_record_log_sound /transcript_log_min_entropy_clear	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1589	/signing_record_transcripts.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1590	elim: rs => [r rs ih] //=.	Performs case analysis or induction, splitting the mathematical proof by constructors or list structure.
1591	move=> [rel sound].	Introduces universally quantified variables or hypotheses into the proof context.
1592	split.	Splits a conjunction or structured goal into independent proof obligations.
1593	+ by apply (signing_record_relation_no_min_entropy_failure md pk r).	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1594	+ by apply ih.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1595	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1596	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1597	lemma fs_with_aborts_no_failure_for_signing_relation	States checked lemma fs_with_aborts_no_failure_for_signing_relation. The following proof script must derive this proposition from prior definitions and lemmas.
1598	md pk m ctx sig tr site :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1599	signing_transcript_relation md pk m ctx sig tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1600	programming_site_matches_transcript md tr site =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1601	! fs_with_aborts_failure md tr site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1602	proof.	Begins the proof script for the preceding proposition.
1603	move=> rel match_site.	Introduces universally quantified variables or hypotheses into the proof context.
1604	rewrite /fs_with_aborts_failure /reprogramming_failure match_site /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1605	by apply (signing_transcript_relation_no_min_entropy_failure	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1606	md pk m ctx sig tr rel).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1607	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1608	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1609	lemma fs_with_aborts_no_failure_for_signing_record md pk r site :	States checked lemma fs_with_aborts_no_failure_for_signing_record. The following proof script must derive this proposition from prior definitions and lemmas.
1610	signing_record_relation md pk r =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1611	programming_site_matches_transcript	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1612	md (signing_record_transcript r) site =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1613	! fs_with_aborts_failure md (signing_record_transcript r) site.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1614	proof.	Begins the proof script for the preceding proposition.
1615	rewrite /signing_record_relation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1616	move=> rel match_site.	Introduces universally quantified variables or hypotheses into the proof context.
1617	by apply (fs_with_aborts_no_failure_for_signing_relation md pk	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1618	(signing_record_message r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.

Line	EasyCrypt source	Mathematical reading
1619	(signing_record_context r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1620	(signing_record_signature r)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1621	(signing_record_transcript r) site).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1622	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1623	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1624	lemma fs_with_aborts_no_failure_for_honest_programming md sk m ctx coins y :	States checked lemma
1625	transcript_challenge_matches	fs_with_aborts_no_failure_for_honest_programming. The following proof script must derive this proposition from prior definitions and lemmas.
1626	(transcript_from_honest_signing md sk m ctx coins) y =>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1627	! fs_with_aborts_failure md	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1628	(transcript_from_honest_signing md sk m ctx coins)	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1629	(programming_site_of_transcript md	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1630	(transcript_from_honest_signing md sk m ctx coins) y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1631	proof.	Begins the proof script for the preceding proposition.
1632	move=> match_y.	Introduces universally quantified variables or hypotheses into the proof context.
1633	rewrite /fs_with_aborts_failure.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1634	rewrite (programming_site_self_no_reprogramming_failure md	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1635	(transcript_from_honest_signing md sk m ctx coins) y match_y).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1636	by rewrite honest_transcript_no_min_entropy_failure.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1637	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1638	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1639	lemma transcript_valid_programming_site_signature_query md tr y :	States checked lemma
1640	transcript_valid md tr =>	transcript_valid_programming_site_signature_query. The following proof script must derive this proposition from prior definitions and lemmas.
1641	programming_site_query (programming_site_of_transcript md tr y) =	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1642	programming_site_query (signature_programming_site_of_transcript md tr y).	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1643	proof.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1644	move=> valid.	Begins the proof script for the preceding proposition.
1645	rewrite /programming_site_query /programming_site_of_transcript	Introduces universally quantified variables or hypotheses into the proof context.
1646	/signature_programming_site_of_transcript /=.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1647	by apply transcript_valid_signature_challenge_query.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1648	qed.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1649	blank	Ends the proof; EasyCrypt has accepted all remaining obligations.
1650	lemma transcript_valid_signature_programming_site_matches md tr y :	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1651	transcript_valid md tr =>	States checked lemma
1652	transcript_signature_challenge_matches tr y =>	transcript_valid_signature_programming_site_matches. The following proof script must derive this proposition from prior definitions and lemmas.
		Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
		Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.

Line	EasyCrypt source	Mathematical reading
1653	<code>programming_site_matches_transcript md tr</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1654	<code>(signature_programming_site_of_transcript md tr y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1655	<code>proof.</code>	Begins the proof script for the preceding proposition.
1656	<code>move=> valid match_y.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1657	<code>rewrite /programming_site_matches_transcript</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1658	<code>/signature_programming_site_of_transcript</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1659	<code>/programming_site_query /programming_site_output /=.</code>	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1660	<code>rewrite -(transcript_valid_signature_challenge_query md tr valid).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1661	<code>move: match_y.</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1662	<code>rewrite -(transcript_valid_signature_challenge_matches md tr y valid).</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1663	<code>by [].</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1664	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1665	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1666	<code>lemma transcript_valid_signature_programming_site_no_reprogramming</code>	States checked lemma <code>transcript_valid_signature_programming_site_no_reprogramming</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1667	<code>md tr y :</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1668	<code>transcript_valid md tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1669	<code>transcript_signature_challenge_matches tr y =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1670	<code>! reprogramming_failure md tr</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1671	<code>(signature_programming_site_of_transcript md tr y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1672	<code>proof.</code>	Begins the proof script for the preceding proposition.
1673	<code>move=> valid match_y.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1674	<code>rewrite /reprogramming_failure.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1675	<code>by rewrite (transcript_valid_signature_programming_site_matches</code>	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1676	<code>md tr y valid match_y).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1677	<code>qed.</code>	Ends the proof; EasyCrypt has accepted all remaining obligations.
1678	<code>blank</code>	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1679	<code>lemma transcript_valid_actual_signature_programming_site_matches md tr :</code>	States checked lemma <code>transcript_valid_actual_signature_programming_site_matches</code> . The following proof script must derive this proposition from prior definitions and lemmas.
1680	<code>transcript_valid md tr =></code>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1681	<code>programming_site_matches_transcript md tr</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1682	<code>(actual_signature_programming_site md tr).</code>	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1683	<code>proof.</code>	Begins the proof script for the preceding proposition.
1684	<code>move=> valid.</code>	Introduces universally quantified variables or hypotheses into the proof context.
1685	<code>rewrite /actual_signature_programming_site.</code>	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1686	<code>apply (transcript_valid_signature_programming_site_matches md tr</code>	Applies an existing theorem, lemma, or proof rule to reduce the current goal to its premises.

Line	EasyCrypt source	Mathematical reading
1687	(transcript_signature_challenge_output tr) valid).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1688	by apply transcript_signature_challenge_output_matches.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1689	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1690	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1691	lemma transcript_valid_actual_signature_programming_site_no_reprogramming	States checked lemma
		transcript_valid_actual_signature_programming_site_no_reprogramming.
		The following proof script must derive this proposition from prior definitions and lemmas.
1692	md tr :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1693	transcript_valid md tr =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1694	! reprogramming_failure md tr (actual_signature_programming_site md tr).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1695	proof.	Begins the proof script for the preceding proposition.
1696	move=> valid.	Introduces universally quantified variables or hypotheses into the proof context.
1697	rewrite /reprogramming_failure.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1698	by rewrite (transcript_valid_actual_signature_programming_site_matches	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1699	md tr valid).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1700	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1701	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1702	lemma valid_forking_pair_signature_programming_site_query md tr1 tr2 y1 y2 :	States checked lemma
		valid_forking_pair_signature_programming_site_query. The following proof script must derive this proposition from prior definitions and lemmas.
1703	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1704	programming_site_query (signature_programming_site_of_transcript md tr1 y1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1705	programming_site_query (signature_programming_site_of_transcript md tr2 y2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1706	proof.	Begins the proof script for the preceding proposition.
1707	move=> fork_pair.	Introduces universally quantified variables or hypotheses into the proof context.
1708	rewrite /programming_site_query	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1709	/signature_programming_site_of_transcript /.	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1710	by apply valid_forking_pair_signature_challenge_query.	Discharges the current goal in one step using the cited simplification, lemma, or automation.
1711	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1712	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1713	lemma valid_forking_pair_programming_site_query md tr1 tr2 y1 y2 :	States checked lemma valid_forking_pair_programming_site_query. The following proof script must derive this proposition from prior definitions and lemmas.
1714	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1715	programming_site_query (programming_site_of_transcript md tr1 y1) =	Part of an equality or definition, identifying two mathematical expressions or assigning a program value.
1716	programming_site_query (programming_site_of_transcript md tr2 y2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1717	proof.	Begins the proof script for the preceding proposition.
1718	move=> fork_pair.	Introduces universally quantified variables or hypotheses into the proof context.
1719	rewrite /programming_site_query /programming_site_of_transcript /.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1720	by apply valid_forking_pair_challenge_query.	Discharges the current goal in one step using the cited simplification, lemma, or automation.

Line	EasyCrypt source	Mathematical reading
1721	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1722	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1723	lemma valid_forking_pair_self_programming_sites_no_reprogramming	States checked lemma valid_forking_pair_self_programming_sites_no_reprogramming. The following proof script must derive this proposition from prior definitions and lemmas.
1724	md tr1 tr2 y1 y2 :	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1725	valid_forking_pair md tr1 tr2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1726	transcript_challenge_matches tr1 y1 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1727	transcript_challenge_matches tr2 y2 =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1728	! reprogramming_failure md tr1	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1729	(programming_site_of_transcript md tr1 y1) /\	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1730	! reprogramming_failure md tr2	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1731	(programming_site_of_transcript md tr2 y2).	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1732	proof.	Begins the proof script for the preceding proposition.
1733	move=> _ match1 match2.	Introduces universally quantified variables or hypotheses into the proof context.
1734	split.	Splits a conjunction or structured goal into independent proof obligations.
1735	+ by apply programming_site_self_no_reprogramming_failure.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1736	+ by apply programming_site_self_no_reprogramming_failure.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1737	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1738	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1739	lemma fs_bound_parts_nonnegative :	States checked lemma fs_bound_parts_nonnegative. The following proof script must derive this proposition from prior definitions and lemmas.
1740	fs_with_aborts_bound_obligation =>	Introduces implication structure: later proof steps may assume the antecedent to prove the conclusion.
1741	0%r <= fs_with_aborts_reprogramming_term /\	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1742	0%r <= fs_with_aborts_min_entropy_term.	Part of an inequality, usually comparing probabilities or bounding a concrete loss term.
1743	proof. by rewrite /fs_with_aborts_bound_obligation. qed.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1744	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1745	lemma fs_with_aborts_bound_obligation_holds :	States checked lemma fs_with_aborts_bound_obligation_holds. The following proof script must derive this proposition from prior definitions and lemmas.
1746	fs_with_aborts_bound_obligation.	Continuation line in the current declaration or proof. Read it with surrounding lines as part of the same mathematical object.
1747	proof.	Begins the proof script for the preceding proposition.
1748	rewrite /fs_with_aborts_bound_obligation.	Rewrites the goal using definitions or previously proved equalities, exposing the intended mathematical normal form.
1749	split.	Splits a conjunction or structured goal into independent proof obligations.
1750	+ by apply fs_with_aborts_reprogramming_term_nonnegative.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1751	+ by apply fs_with_aborts_min_entropy_term_nonnegative.	Solves a proof branch produced by a previous split, transitivity, or case analysis.
1752	qed.	Ends the proof; EasyCrypt has accepted all remaining obligations.
1753	blank	Blank separator. It marks a boundary between declarations, proof blocks, or tactic phases.
1754	end HAETAE_ROM_Programming.	Closes the current theory or module block.